
ENGIAI: CAPABILITY-BASED EVALUATION OF TOOL-CONNECTED LLM AGENTS FOR ENGINEERING DESIGN

PREPRINT

 **Gioele Molinari***

IDEAL
Chair of Artificial Intelligence in Engineering Design
ETH Zurich
Zurich, Switzerland
gioelemo@ethz.ch

 **Florian Felten**

IDEAL, ETH Zurich, Zurich, Switzerland
herve.review, Brussels, Belgium

 **Soheyl Massoudi**

IDEAL
Chair of Artificial Intelligence in Engineering Design
ETH Zurich
Zurich, Switzerland

 **Mark Fuge**

IDEAL
Chair of Artificial Intelligence in Engineering Design
ETH Zurich
Zurich, Switzerland

ABSTRACT

Engineering-agent systems are proliferating, but differences in tasks, tools, and success criteria make demonstrations difficult to compare and failures difficult to diagnose. We introduce a capability-based evaluation framework for tool-connected engineering agents. The framework separately evaluates workflow execution, retrieval-assisted parameter selection, high-performance computing (HPC) orchestration, and training-code authoring using execution traces and resulting engineering artifacts. We evaluate four LLM backends on the EngiBench Beams2D and Photonics2D problems using ENGIAI, a LangGraph reference implementation. On Beams2D, the two proprietary models complete 96–97% of workflow tasks, compared with 55–78% for the two open-source models. Workflows requiring tool-based decision-making perform worse on Photonics2D, reaching 20–53% completion. Indexed retrieval improves parameter selection. For HPC orchestration, Gemini-3-Flash completes every tested pipeline, whereas GPT-5-mini completes 50–70% through the final evaluation step. When the agents must write the training code themselves, they fill small gaps reliably but diverge as larger regions are left open. In the open-synthesis tier, both proprietary models select conditional variational autoencoders instead of the reference conditional Generative Adversarial Network (cGAN); Gemini-3-Flash achieves lower maximum mean discrepancy than the reference in seven of ten Beams2D trials. The results support capability-specific evaluation, with separate scores for distinct skills, rather than assessment through successful end-to-end demonstrations alone. The framework provides common evaluation dimensions for identifying failure mechanisms and structuring comparisons of models, agent architectures, and tool interfaces.

Keywords engineering agents · agent evaluation · large language models · engineering design · high-performance computing

1 Introduction

Engineering design workflows span problem definition, concept generation, analysis and optimization, and preparation for manufacturing [52, 70]. These stages are usually supported by separate software tools and require users to transfer data and decisions between them. Deep generative models support engineering design-space exploration and

*Corresponding author.

optimization, including quality- and diversity-aware generation [10] and design under manufacturing uncertainty [11]. Their use adds further workflow steps, including data preparation, model training, and evaluation [58].

Large Language Models (LLMs) agents can coordinate external tools from natural-language instructions by selecting an appropriate tool, generating its arguments, observing its output, and using that output in subsequent decisions [74, 62]. However, recent benchmarks report substantial failures in multi-step tool use and scientific workflows [75, 12]. It therefore remains unclear whether current agents can perform these operations reliably across complete engineering workflows, and for which steps they break down.

LLM-agent systems are being proposed for a growing range of engineering activities, including conceptual design, simulation, optimization, manufacturing, and design coordination [45, 55, 9, 40]. However, these systems generally use different engineering problems, tool interfaces, prompts, stopping conditions, and success criteria. A successful end-to-end demonstration therefore does not reveal whether performance arose from the underlying LLM, the orchestration logic, retrieved information, the tool interface, or the engineering program being executed. These differences also prevent direct comparison across agent systems and make it difficult to translate individual demonstrations into general development guidance.

Here, we use the term *agent harness* for the components that connect an LLM to an engineering workflow: prompts, tool schemas, routing and state management, retrieval, execution control, and output validation. Existing benchmarks evaluate several of these components in general-purpose settings, but they do not establish how their failures propagate into engineering artifacts, remote training jobs, or trained models. A useful engineering-agent evaluation must therefore assess both the execution trace and the resulting engineering outcome, while separating the capabilities required at different stages of the workflow.

This paper introduces a benchmark suite for evaluating LLM-driven engineering workflows, together with ENGIAI, a multi-agent reference implementation that operationalizes it. Accessible through a web-based interface, ENGIAI coordinates multiple engineering tools through natural-language interaction, with a modular architecture that allows new capabilities to be added via additional agents or tool APIs. The benchmark suite and reference implementation together address the following research question:

How can tool-connected LLM agents be evaluated by capability, and what do these evaluations reveal about failures in workflow execution, document-grounded parameter selection, remote training-job orchestration, and training code authoring?

We evaluate these four capabilities separately so that failures can be attributed to a specific part of the workflow. This paper makes four contributions. First, we formulate an evaluation framework that separates four capabilities of tool-connected engineering agents: workflow execution, retrieval-assisted parameter selection, remote training-job orchestration, and training-code authoring. Second, we instantiate this framework as a benchmark that combines controlled prompt variations and retrieval ablations with capability-specific metrics, scoring both tool traces and resulting engineering artifacts. Third, we apply the benchmark to four LLM backends and two engineering problems, identifying failures in semantic grounding, conditional decisions, long-horizon completion, checkpoint compatibility, sampling behavior, and trained-model quality. Fourth, we provide ENGIAI as an open reference implementation for comparing model backends within its multi-agent architecture. Applying the same tasks and scorers to another agent architecture requires a compatible tool, trace, and artifact interface (video demonstration).

2 Related Work

Pahl and Beitz [52] organize engineering design into task clarification, conceptual design, embodiment design, detail design, and preparation of production documentation. Other models represent design and development processes differently [70], but iteration is common across them: analysis, testing, and new information can require earlier decisions to be revisited [71]. Decision-based design provides a complementary view in which design involves selecting alternatives under objectives, constraints, and uncertainty [26].

ML-augmented design adds infrastructure for training surrogate and generative models, as provided by systems such as EngiOpt [19]. It also introduces coordination among heterogeneous software tools, which Jiang *et al.* [31] describe within their Intelligent Design 4.0 framework.

The following subsections review related work on multi-agent orchestration, engineering-tool integration and manufacturing, retrieval-augmented engineering, agent benchmarking, and machine-learning code authoring.

2.1 Multi-agent orchestration

Recent surveys describe the use of LLM-based agents for multi-step tasks across scientific domains [1, 24]. General-purpose frameworks such as AutoGen [69], CrewAI [13], the OpenAI Agents SDK [47], and LangGraph [34] support communication and tool routing among multiple agents. The AI Co-Scientist applies structured agent coordination to research tasks [23]. Jiang *et al.* [31] place such systems within their Intelligent Design 4.0 framework for multi-agent design automation.

Engineering applications have used agent systems for topology optimization and finite-element analysis [45, 56, 15], aerospace structural analysis [55], automotive and product design [17, 9], material selection [76], iterative battery-pack design [72], mechatronics [66], and design-state coordination across LLM backends [40]. TopOptAgents [53] coordinates six agents to formulate, execute, and refine a topology-optimization problem. These systems address specific design, analysis, or coordination tasks. Our experiments additionally evaluate document retrieval, remote training-job management, and training-code authoring within one tool-connected workflow.

2.2 Tool integration and manufacturing

Tool interfaces determine which engineering operations an agent can perform. The Model Context Protocol (MCP) provides a common interface for external services [4], although tool-selection accuracy declines as tasks and tool sets become more complex [67]. Jadhav and Farimani [30] use a hierarchical agent system to monitor fused-deposition-modeling processes. EngiBench and EngiOpt provide Python interfaces for engineering problems, optimization, and generative design [19]. ENGIAI connects these interfaces so that the benchmark can evaluate both the selected tools and the engineering outputs they produce.

2.3 Retrieval-augmented engineering

RAG supplies an LLM with passages retrieved from an external document collection, reducing its reliance on parametric knowledge alone [35, 22]. This is particularly relevant when an engineering workflow requires numerical parameters from technical documents, because an incorrect value can produce an invalid or misleading design. Chandrasekhar *et al.* [6] combine Llama2-7B with indexed Additive Manufacturing (AM) papers and report performance competitive with GPT-4 on metal additive-manufacturing questions. Khanghah *et al.* [33] apply multimodal RAG to zero-shot anomaly detection in laser powder bed fusion and report a 12% improvement in classification accuracy over non-retrieval baselines. These studies evaluate question answering or classification. They do not examine whether an agent can retrieve numerical parameters and use them correctly in subsequent simulation or design-tool calls. Our RAG experiment evaluates this latter task.

2.4 Agent benchmarking

Surveys of agent evaluation distinguish capabilities such as tool selection, argument accuracy, multi-step planning, and task completion [43]. ACEBench [8], ToolSandbox [38], BFCL [54], and ToolBench [57] focus primarily on tool calls and report performance differences between open-source and proprietary models. AgentBench [36], AgentBoard [39], and ScienceAgentBench [12] evaluate longer reasoning and execution sequences across multiple environments, primarily through single-agent tasks. For multi-turn professional tasks, τ -bench [75] reports task-success rates below 50% even for state-of-the-art models.

Engineering-specific benchmarks address more specialized capabilities. FDM-Bench [18] evaluates LLMs on fused-deposition-modeling tasks, while Zhou *et al.* [77] introduce a hierarchical engineering benchmark and show that current models struggle with open-ended modeling problems. These benchmarks cover important aspects of tool use, reasoning, and engineering knowledge, but they do not jointly evaluate a multi-agent workflow that combines simulation, document retrieval, and remote HPC execution.

ScienceAgentBench is especially relevant because it decomposes scientific workflows into executable tasks and evaluates generated programs, execution results, and cost [12]. EngDesign evaluates functional engineering outputs through simulation across nine engineering domains [25]. These studies establish task-level scientific evaluation and simulation-based engineering verification, respectively. Our benchmark examines a different combination: failures propagated across workflow execution, document-grounded parameter selection, remote training-job management, and training-code authoring.

Our benchmark adds three requirements that arise when these capabilities are connected. First, numerical outputs from one tool must be used correctly in later operations, such as selecting an export branch from a simulation result. Second, the agent must distinguish between competing plausible parameter values and preserve their numerical precision. Third,

it must retain separate parameter sets across sequential tool calls. We evaluate these behaviors using both the tool-call trace and the resulting engineering artifact or trained model.

2.5 Machine-learning engineering agents

Software-engineering benchmarks such as SWE-bench evaluate whether agents can resolve issues in existing repositories [32]. A related group of benchmarks evaluates agents that write and revise machine-learning code. MLEAgentBench tests iterative modifications to training programs and reports near-zero success on its more recent datasets [29]. MLE-bench evaluates agents through Kaggle competitions [5], while MLGym and RE-Bench cover more open-ended machine-learning research tasks [44, 68]. MLGym finds that agents can tune hyperparameters more reliably than they can implement model architectures that differ from the supplied baseline. The AI Scientist generates and iterates on complete research ideas, but evaluates the resulting paper with an automated reviewer rather than evaluating the engineering output of the trained model [37]. AutoML-Agent instead coordinates specialist LLM agents across stages of an AutoML workflow [64].

These studies show that agents can be evaluated through the code they produce and, in several cases, through the performance of the resulting model. Our benchmark applies this approach to generative models of engineering designs. It separates training-code authoring into progressively more open tasks. The configuration tier modifies parameters in an otherwise fixed program; the code-completion tiers require selected regions to be implemented; and the open-synthesis tier supplies only the data and evaluation interfaces, allowing the agent to select and implement a model architecture. Here, “open synthesis” means selecting an architecture that is not prescribed by the supplied reference program; it does not imply an algorithm that is novel to the research literature.

The authored program must then be submitted to an HPC/SLURM cluster, complete training, and produce a model whose generated designs are scored by a functional engineering evaluator. The benchmark therefore measures both code authoring and completion of the remote training workflow. Among the studies reviewed above, this combination of progressively open training-code authoring, functional evaluation of generated engineering designs, and cluster-level execution is not otherwise assessed.

The reviewed studies cover different parts of tool-connected engineering workflows. ENGIAI implements seven system capabilities: multi-agent orchestration, physics simulation and topology optimization, document retrieval, manufacturing export, remote HPC orchestration, training-code authoring, and structured benchmarking. We organize the experiments into four evaluation dimensions: workflow execution, retrieval-assisted parameter selection, HPC orchestration, and training-code authoring. Each evaluation examines both the execution trace and the resulting engineering artifact. Table 1 compares ENGIAI with the systems and benchmarks reviewed above.

3 Methods

3.1 System Architecture

The ENGIAI system follows a hierarchical supervisor pattern (Figure 1). A central supervisor receives each user prompt, classifies its intent, and routes it to the appropriate specialist agent. Each specialist is implemented as a LangGraph state machine that receives a task, invokes its domain-specific tools, and returns the result to the supervisor.

We use this decomposition to limit the number of tools exposed to each specialist. This design choice is motivated by evidence that LLMs become less accurate at tool selection as the number of available tools increases [67]. The present study does not compare this supervisor architecture with a single agent having access to the complete tool set.

The supervisor can invoke several specialists sequentially. For example, it can first route a request to the RAG agent to retrieve parameters and then pass those parameters to the engineering agent. LangGraph stores the workflow state, binds tools to agents, supports human-interruption points, and persists checkpoints [7]. ENGIAI terminates a workflow when it detects repeated rerouting.

A typical design session illustrates the pattern: when a user requests a topology optimization, the supervisor routes to the *engineering agent*, which calls *create_problem* → *optimize_design* → *simulate_design* → *render_design* (or *convert_design_to_stl* for manufacturing export). If the user first asks about a parameter from a paper, the supervisor routes to the *RAG agent*, which retrieves the relevant information, then re-routes to the *engineering agent* with the extracted values.

Table 1: Representative selection of LLM-relevant engineering capabilities covered by ENGIAI versus prior work, grouped into *Capabilities*, *Infrastructure*, and *Evaluation*. ✓ = supported, × = not supported. Systems are split into engineering agent systems (top) and evaluation benchmarks (bottom).

	Capabilities					Infra.	Eval.
	Multi-Agent	RAG	Simulation	Mfg. Export	Code Gen.	HPC	Benchmark
<i>Engineering Agent Systems</i>							
DSG-MAS [40]	✓	×	✓	×	✓	×	✓
MechAgents [45]	✓	×	✓	×	✓	×	×
FEAGPT [56]	✓	✓	✓	×	✓	×	×
LLM3DPrint [30]	✓	×	×	✓	×	×	×
DUCTILE [55]	×	×	✓	×	✓	×	✓
AMGPT [6]	×	✓	×	×	×	×	×
Wang et al. [66]	✓	×	✓	×	✓	×	×
DesignAgent [9]	✓	✓	✓	×	×	×	×
ALL-FEM [15]	✓	✓	✓	×	✓	×	✓
TopOptAgents [53]	✓	×	✓	×	✓	×	×
<i>Evaluation Benchmarks</i>							
EngiBench/EngiOpt [19]	×	×	✓	×	×	×	✓
EngiBench [77]	×	×	×	×	×	×	✓
FDM-Bench [18]	×	×	×	×	×	×	✓
EngDesign [25]	×	×	✓	×	×	×	✓
General benchmarks [8, 38, 54, 57, 67, 36, 39, 12, 75, 27, 14]	×	×	×	×	✓	×	✓
Code-authoring benchmarks [44, 29, 5, 32, 64, 68]	✓	✓	×	×	✓	×	✓
ENGIAI (ours)	✓	✓	✓	✓	✓	✓	✓

3.2 Agent Capabilities

Table 2 summarizes the seven specialist agents and their tools. We organize them by their role in the design-to-manufacturing lifecycle.

Engineering agent. The core design agent drives topology optimization, simulation, ML-based inverse design, and STL export through the EngiBench/EngiOpt [19] stack. EngiBench provides a unified Python API for accessing engineering design problems across multiple physics domains; EngiOpt extends it with ML-based generative design algorithms (*e.g.*, conditional Generative Adversarial Network (cGAN)s, diffusion models) that can be trained and evaluated against dataset baselines. Both model families are established for engineering design: cGANs [42] have been applied to structural topology optimization [46], and diffusion models were subsequently shown to outperform them on performance- and manufacturability-aware topology generation [41].

Information retrieval agents. Three agents provide complementary retrieval capabilities. The *RAG agent* and *ArXiv agent* share the MMORE (Multimodal Open Retrieval Engine) [61] framework, which indexes and retrieves information from heterogeneous document types (PDFs, images, tables)—the former for document Q&A over user-indexed collections, the latter for fetching and indexing papers from arXiv. The *search agent* provides web search via the Tavily API.

Infrastructure agents. The *HPC agent* manages SLURM jobs on remote clusters via SSH. The *CLI agent* executes local shell commands (with a whitelisted set of GUI applications and human-in-the-loop confirmation step before running commands), and the *3D printer agent (Prusa)* controls 3D printers through a dedicated MCP [4] server.

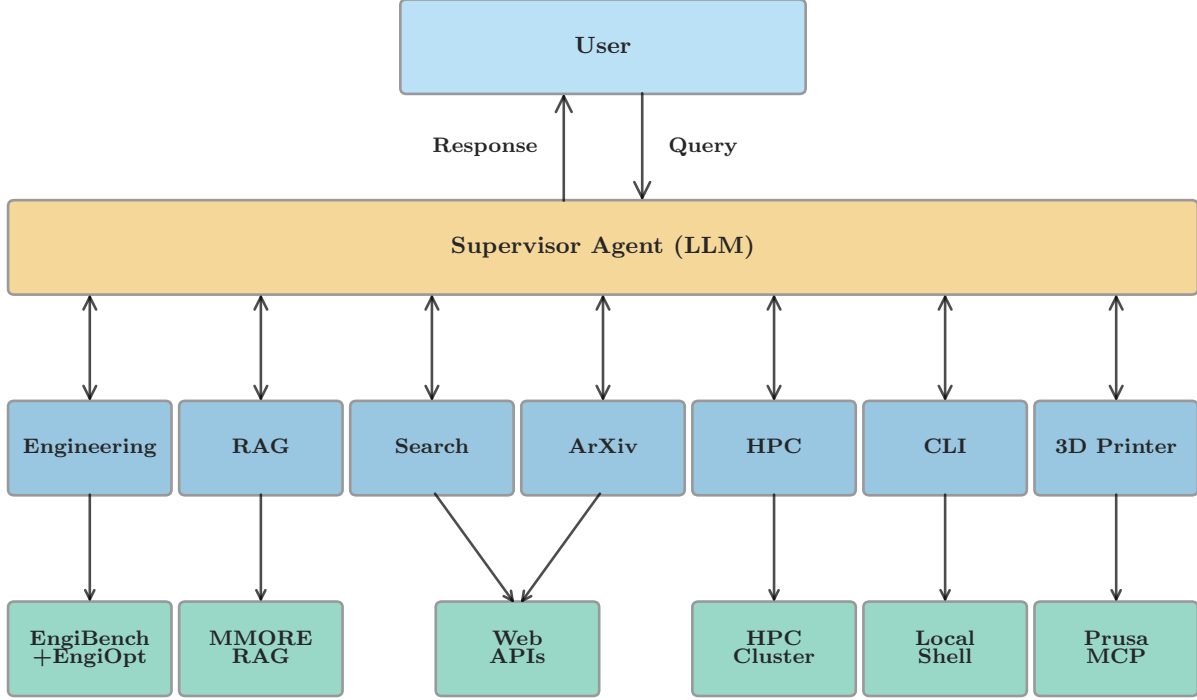


Figure 1: Multi-agent architecture. From top to bottom: the user interface, the orchestration layer (supervisor agent routing prompts), the specialization layer (seven domain agents), and the execution layer (external services and tools).

Table 2: ENGIAI multi-agent architecture and tool integration.

Agent	Primary Function	# Tools
Supervisor	Request routing	–
Engineering	Optimization, ML, STL export	14
Search	Web retrieval	1
RAG	Document Q&A	5
ArXiv	Paper retrieval	5
HPC	Remote job submission	8
CLI	Command execution	2
3D Printer (Prusa MCP)	3D printer control	8

3.3 Experimental Setup

3.3.1 Evaluation Problems

We evaluate on two EngiBench [19] problems spanning different physics domains.

Beams2D. A 50×100 cantilever beam topology optimization problem minimizing compliance subject to a volume fraction constraint [63, 2]. This well-studied structural optimization benchmark provides ground-truth solutions for evaluating agent correctness. Key parameters are volume fraction (*volfrac*), force distance (*forcedist*), and filter radius (*rmin*).

Photonics2D. A 120×120 photonic device inverse design problem maximizing electromagnetic field overlap for specified wavelengths, testing whether agent capabilities transfer across physics domains.

3.3.2 Models and Configuration

We evaluate two proprietary models, GPT-5-mini (gpt-5-mini-2025-08-07) [49] and Gemini-3-Flash (gemini-3-flash-preview), and two open-source 4B-parameter models served through Ollama, Qwen3-4B (qwen3:4b-instruct-2507-q8_0) [73] and Qwen3.5-4B (qwen3.5:4b-q8_0). This selection permits a comparison

Table 3: Operationalization of the capability-based evaluation framework. Each evaluation varies the information, instructions, or implementation responsibility given to the agent and collects evidence needed to distinguish execution, grounding, interface, and outcome failures.

Evaluation dimension	Experimental contrast	Evidence collected	Failures distinguished
Workflow execution	Seven prompt styles vary explicitness, ambiguity, distractors, derived parameters, conditional decisions, and retention of multiple parameter sets.	Expected and observed tool calls, task completion, parameter validation, and the resulting engineering design.	Missing or redundant calls, failure to request clarification, incorrect numerical values, branch inversion, and interference between parameter sets.
Retrieval-assisted parameter selection	RAG-on and Empty RAG hold tool access constant while varying indexed document content. RAG-off separately verifies behavior without retrieval access.	Retrieval invocation, selected parameter values, and the values passed to downstream design-tool calls.	Failure to retrieve, reliance on unsupported defaults, incomplete parameter sets, and incorrect values passed downstream.
HPC orchestration	Explicit prompts enumerate the required operations, whereas natural prompts require the agent to infer the remote training workflow.	SLURM configuration, script acceptance, job submission, monitoring, final evaluation, and extracted model metrics.	Invalid configurations, failed submission or monitoring, premature workflow termination, and omission of the final evaluation.
Training-code authoring	T1–T6v2 progressively increase the code authored by the agent. Canonical and functional evaluators separate reference-interface compatibility from functional model behavior.	Syntax and structural validation, local dry run, SLURM acceptance, job completion, checkpoint loading, sampler behavior, and MMD.	Syntax or interface errors, execution failure, checkpoint incompatibility, degenerate or condition-insensitive sampling, and poor trained-model quality.

between proprietary and locally served models and between two Qwen generations at the same parameter count. All models support structured tool calls. We set the sampling temperature to zero and the API seed to 42. Each workflow model–style cell contains 15 runs: three random seeds and five dataset samples. Beams2D uses all seven prompt styles; Photonics2D uses three. Appendix A reports the compute and inference settings.

3.4 Benchmark Design

The benchmark addresses four questions: (1) *workflow execution*—can agents reliably chain engineering tools when prompts impose different cognitive demands? (2) *retrieval-augmented parameter selection*—does document retrieval improve the selection of engineering parameters? (3) *HPC orchestration*—can agents submit, monitor, and evaluate long-running training jobs on remote infrastructure? and (4) *training-code authoring*—can agents write and execute training code rather than only run a supplied program? Sections 3.4.1–3.4.4 define the tasks and success criteria for each evaluation.

Table 3 summarizes how each evaluation is conducted, what evidence it records, and which types of failure it can identify.

3.4.1 Workflow Evaluation

We define seven prompt styles spanning distinct cognitive demands (Table 4; representative prompts in Appendix C). The first two establish baselines: FULL provides all numerical parameters for a direct optimize→simulate→render pipeline, while NATURAL uses qualitative descriptions (e.g., “lightweight” instead of $\text{volfrac} = 0.2$), requiring the agent to detect ambiguity and request human clarification.

The five W-styles extend FULL with an STL export step. Each modifies only the export instructions while keeping the optimization and simulation steps identical, isolating the effect of instruction phrasing on task completion. The styles target three categories of failure mode that standard tool-calling benchmarks do not capture.

The first category is *numerical fidelity*: can the agent propagate precise values from prompt to tool call? W-RAND tests this directly with explicit randomized export parameters (density threshold, XY scale, Z extrusion, mirror flag).

W-DERIVED raises the bar by specifying parameters as derivation rules (*e.g.*, “use the volume fraction as the density threshold”), requiring arithmetic over values the agent passed to earlier tools.

The second is *semantic grounding*: can the agent select the correct value when the prompt contains competing alternatives? W-DISTRACT presents two plausible values for the same parameter—one in a preview context, one for manufacturing export—both of which the tool accepts without error. Only the parameter-validation scorer distinguishes correct from incorrect choices, making this failure mode invisible to benchmarks that check only function signatures.

The third is *stateful reasoning*: can the agent condition its actions on runtime information? W-COND requires the agent to read a compliance value from *simulate_design* and select the corresponding parameter branch via an if/then rule. W-MULTI requires two sequential export calls with entirely different parameter sets, testing whether the agent can maintain distinct configurations in working memory rather than merging or dropping one.

Table 4: Workflow evaluation prompt styles. [†]Randomized STL parameters (*mirror_y*, *scale_xy*, *scale_z*, *threshold*); validated within ± 0.05 absolute tolerance on each parameter value to account for LLM rounding. [‡]STL parameters derived from optimization inputs.

Style	Input	Capability	Sequence
FULL	All num. params	Direct tool use	opt→sim→ren
NATURAL	Text only	Ambiguity detect.	ask_human
W-RAND	+ rand. STL [†]	Instr. adherence	opt→sim→stl
W-DERIVED	+ deriv. rules [‡]	Arithmetic	opt→sim→stl
W-DISTRACT	+ competing vals [†]	Semantic disamb.	opt→sim→stl
W-COND	+ branching [†]	Conditional	opt→sim→stl
W-MULTI	+ two exports [†]	Working memory	opt→sim→stl→stl

3.4.2 RAG Evaluation

We design four fixed prompts with different retrieval requirements (Table 5; full text in Appendix C). Unlike the workflow benchmarks, which sample from EngiBench datasets, the RAG evaluation instructs the agent to retrieve specific optimization parameters from indexed research papers and use them in a design task. The prompts range from retrieving a single parameter (P0) to combining parameters from two papers (P3).

Table 5: RAG evaluation prompts with escalating retrieval difficulty. Scoring weights are given in Table 12.

ID	Source	Target Parameters		
		volfrac	forcedist	rmin
P0	EngiBench paper [19]	0.35	—	—
P1	EngiBench API ex. [19]	0.70	0.30	—
P2	SOPTX paper [28]	0.40	—	6.0
P3	EngiBench [19] + SOPTX [28]	0.70	0.30	6.0

We evaluate under three conditions to isolate the contribution of document retrieval. RAG-on provides reference documents indexed and retrievable via *search_documents*. RAG-off removes retrieval tools entirely, forcing the agent to rely on parametric knowledge. Empty RAG is the critical ablation: the retrieval tools are available but the index is empty, testing whether the mere availability of a retrieval tool changes agent behavior—for example, by inducing the agent to trust empty results and omit parameters it would otherwise guess from training data.

To ensure the only retrieval path is *search_documents*, we disable the web search and ArXiv agents for all RAG evaluation runs.

The RAG service (MMORE) synthesizes retrieved passages at temperature 0.8; this applies only to retrieval-answer synthesis, and the evaluated agent remains at temperature zero. Additional details of the RAG service are provided in Appendix A.

3.4.3 HPC Training Evaluation

The HPC benchmark requires four operations. The agent must generate a SLURM script with the requested configuration, submit it, monitor the job until it terminates, and evaluate the trained model. Evaluation uses the EngiOpt metrics

defined in Appendix B.5: Maximum Mean Discrepancy (MMD), Determinantal Point Process (DPP), Ratio of Violated Constraints (RVC), and optimality gaps Intermediate Optimality Gap (IOG), Cumulative Optimality Gap (COG), and Final Optimality Gap (FOG).

We define 10 prompts per style (seeds 1–10) with 100 fixed training epochs, using two prompt formats (Appendix C): EXPLICIT (step-by-step instructions with tool names) and NATURAL (high-level description requiring the agent to infer the workflow). We report results for the cGAN algorithm; diffusion model results are provided in Appendix D.2.

3.4.4 Training-Code Authoring

The training-code benchmark extends the HPC benchmark by requiring the agent to write part or all of the training program. T1 uses a fixed program and varies only its launch configuration. T2–T5 remove progressively larger code regions from the EngiOpt cGAN program. T6 removes the model and training implementation and supplies only the required command-line and evaluation interfaces. The evaluated implementation of this final tier is denoted T6v2. Neither construction is specific to the cGAN: T2–T5 blank regions of any supplied reference implementation, a conditional diffusion template is also provided, and T6 imposes no algorithm at all. Figure 2 lists the code supplied at each tier. Representative prompts are provided in Appendix C.4.

T1 Configure & launch	T2 Plumbing	T3 Training loop	T4 Architecture	T5 Whole module	T6 Open synthesis
Imports & args	Imports & args	Imports & args	Imports & args	Imports & args	write ANY algorithm (GAN / diffusion / novel) no engiopt skeleton; only the sample(c) contract is fixed, scored on outcome
Generator (NetG)	Generator (NetG)	Generator (NetG)	Generator (NetG)	Generator (NetG)	
Discriminator (NetD)	Discriminator (NetD)	Discriminator (NetD)	Discriminator (NetD)	Discriminator (NetD)	
Dataloader	Dataloader	Dataloader	Dataloader	Dataloader	
Sampling helper	Sampling helper	Sampling helper	Sampling helper	Sampling helper	
Training loop (loss, optimizers)	Training loop (loss, optimizers)	Training loop (loss, optimizers)	Training loop (loss, optimizers)	Training loop (loss, optimizers)	
Logging / device	Logging / device	Logging / device	Logging / device	Logging / device	
Checkpoint save	Checkpoint save	Checkpoint save	Checkpoint save	Checkpoint save	
Launch command (problem, epochs, seed)	Launch command (problem, epochs, seed)	Launch command (problem, epochs, seed)	Launch command (problem, epochs, seed)	Launch command (problem, epochs, seed)	

Figure 2: Training-code authoring tiers. Shaded blocks are written by the agent; unshaded blocks are supplied. T1 configures a fixed program, T2–T5 remove progressively larger code regions, and T6 supplies only the required interfaces. The data loader is supplied in T1–T5.

We evaluate checkpoints in two ways. The canonical evaluator loads the checkpoint with the reference cGAN class. The functional evaluator loads the class and sampling function written by the agent, allowing architectures that do not match the reference class. Both evaluators compute MMD from generated designs. In the functional evaluation, a sampler receives zero model-quality credit only if its outputs are both nearly constant across latent samples and insensitive to changes in conditioning.

3.5 Scoring Methodology

The workflow score combines design quality (65%), tool use (20%), and task completion (15%). Design quality includes geometric agreement, engineering objectives and constraints, and, where applicable, manufacturability (2D connectivity, 3D watertightness). Tool use compares the observed calls with the expected calls. Task completion records whether all required operations succeeded. Appendix B defines each term and reports its weight.

Design quality receives the largest weight (65%) because the benchmark primarily evaluates the engineering output. Tool use accounts for 20% and distinguishes correct but inefficient traces by penalizing missing or redundant calls. Task completion accounts for the remaining 15% and records whether the agent executed the full requested sequence. Appendix E reports the individual sub-scores, allowing the results to be examined under alternative weighting schemes.

Two limitations affect the interpretation of design quality. First, the watertightness sub-score is low across all models because the current threshold-based STL extraction produces non-manifold meshes. This reflects a limitation of the extraction pipeline rather than agent capability, as noted in Appendix Tables 18 and 19. Second, the manufacturability sub-scores apply only to prompt styles that request STL export. Design quality should therefore be compared within each prompt style rather than averaged across styles.

For evaluating generative models (HPC training benchmark), we additionally report global distribution metrics—MMD for dataset fidelity and DPP for design diversity—as well as engineering performance metrics such as RVC and optimality gaps (IOG, COG, FOG), following Felten *et al.* [19]; full definitions are given in Appendix B.5.

RAG Scoring The RAG evaluation employs a gated scoring mechanism: parameter accuracy dimensions only contribute to the score when the agent invokes the *search_documents* tool, preventing credit for values reached through alternative means. The score combines per-parameter accuracy with a retrieval indicator (Appendix B.2).

HPC Workflow Scoring The HPC training benchmark scores workflow orchestration rather than design quality. The primary metric is a weighted composite of step completion (70%), configuration correctness (15%), and evaluation metric extraction (15%). Appendix B.3 provides the full formulation.

Training-Code Scoring The training-code score assigns 55% to model quality, 15% to code validation, 10% each to cluster-job completion and the one-epoch local dry run, and 5% each to syntax validity and SLURM acceptance. These terms are not independent. Model quality is scored only when the trial’s own training job completes, so the largest term requires a full run that produces a working model, and the smaller terms record which intermediate checks the agent passed along the way. For a scorable model, the model-quality term is $\min(1, \text{MMD}_{\text{ref}}/\text{MMD}_{\text{agent}})$. The reference and agent models are matched by seed because reference MMD varies substantially across seeds. Appendix B.4 defines the gates and complete score; Appendix B.5 defines the generative-model metrics.

4 Results

Table 6: Workflow evaluation results for Beams2D (mean $\pm$ std). TC = Task Completion rate, CO = Combined Overall score. **Bold** = best model per metric per row.

Style	GPT-5-mini		Gemini-3-Flash		Qwen3-4B		Qwen3.5-4B	
	TC $\uparrow$	CO $\uparrow$	TC $\uparrow$	CO $\uparrow$	TC $\uparrow$	CO $\uparrow$	TC $\uparrow$	CO $\uparrow$
FULL	1.00 $\pm$ 0.00	0.65 $\pm$ 0.07	0.93 $\pm$ 0.25	0.69 $\pm$ 0.10	0.00 $\pm$ 0.00	0.45 $\pm$ 0.06	0.73 $\pm$ 0.44	0.59 $\pm$ 0.11
NATURAL	0.87 $\pm$ 0.34	0.82 $\pm$ 0.30	1.00 $\pm$ 0.00	0.88 $\pm$ 0.23	0.00 $\pm$ 0.00	0.29 $\pm$ 0.05	0.33 $\pm$ 0.47	0.48 $\pm$ 0.32
W-RAND	1.00 $\pm$ 0.00	0.66 $\pm$ 0.07	1.00 $\pm$ 0.00	0.65 $\pm$ 0.09	1.00 $\pm$ 0.00	0.64 $\pm$ 0.07	1.00 $\pm$ 0.00	0.69 $\pm$ 0.06
W-DERIVED	1.00 $\pm$ 0.00	0.65 $\pm$ 0.07	1.00 $\pm$ 0.00	0.64 $\pm$ 0.07	0.47 $\pm$ 0.50	0.53 $\pm$ 0.08	0.85 $\pm$ 0.36	0.66 $\pm$ 0.10
W-DISTRACT	1.00 $\pm$ 0.00	0.63 $\pm$ 0.06	1.00 $\pm$ 0.00	0.63 $\pm$ 0.08	1.00 $\pm$ 0.00	0.61 $\pm$ 0.06	0.93 $\pm$ 0.25	0.66 $\pm$ 0.09
W-COND	0.93 $\pm$ 0.25	0.66 $\pm$ 0.10	0.87 $\pm$ 0.34	0.65 $\pm$ 0.10	0.40 $\pm$ 0.49	0.57 $\pm$ 0.12	0.60 $\pm$ 0.49	0.62 $\pm$ 0.13
W-MULTI	0.93 $\pm$ 0.25	0.66 $\pm$ 0.09	1.00 $\pm$ 0.00	0.70 $\pm$ 0.07	1.00 $\pm$ 0.00	0.65 $\pm$ 0.08	1.00 $\pm$ 0.00	0.69 $\pm$ 0.06
Average	0.96	0.68	0.97	0.69	0.55	0.53	0.78	0.63

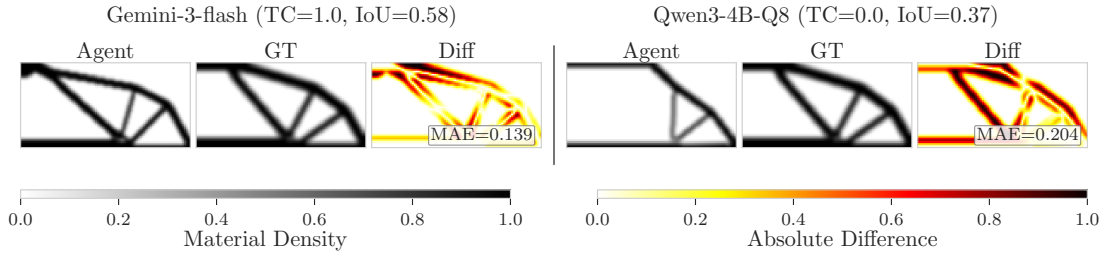


Figure 3: Design comparison for the W-COND style on the same problem instance (Beams2D, seed 3, example 3). Each group shows the agent-generated design (left), ground truth (center), and pixelwise absolute difference (right). Gemini-3-Flash: TC = 1.0 and IoU = 0.58. Qwen3-4B: TC = 0.0 and IoU = 0.37.

Table 7: Workflow evaluation results for Photonics2D (mean $\pm$ std). TC = Task Completion rate, CO = Combined Overall score. **Bold** = best model per metric per row. Gray text shows Beams2D results on the same workflow styles for comparison.

Style	GPT-5-mini		Gemini-3-Flash		Qwen3-4B		Qwen3.5-4B	
	TC $\uparrow$	CO $\uparrow$	TC $\uparrow$	CO $\uparrow$	TC $\uparrow$	CO $\uparrow$	TC $\uparrow$	CO $\uparrow$
W-RAND	1.00$\pm$0.00	0.53 $\pm$ 0.01	1.00$\pm$0.00	0.60$\pm$0.02	1.00$\pm$0.00	0.56 $\pm$ 0.02	1.00$\pm$0.00	0.55 $\pm$ 0.01
W-DISTRACT	1.00$\pm$0.00	0.52 $\pm$ 0.02	1.00$\pm$0.00	0.55 $\pm$ 0.03	1.00$\pm$0.00	0.52 $\pm$ 0.02	1.00$\pm$0.00	0.57$\pm$0.05
W-COND	0.40 $\pm$ 0.49	0.47 $\pm$ 0.08	0.53$\pm$0.50	0.54$\pm$0.09	0.20 $\pm$ 0.40	0.43 $\pm$ 0.07	0.47 $\pm$ 0.50	0.49 $\pm$ 0.08
<i>Average</i>	<i>0.80</i>	<i>0.51</i>	<i>0.84</i>	<i>0.56</i>	<i>0.73</i>	<i>0.50</i>	<i>0.82</i>	<i>0.54</i>
<i>Beams2D (same styles, for comparison)</i>								
W-RAND	1.00$\pm$0.00	0.66 $\pm$ 0.07	1.00$\pm$0.00	0.65 $\pm$ 0.09	1.00$\pm$0.00	0.64 $\pm$ 0.07	1.00$\pm$0.00	0.69$\pm$0.06
W-DISTRACT	1.00$\pm$0.00	0.63 $\pm$ 0.06	1.00$\pm$0.00	0.63 $\pm$ 0.08	1.00$\pm$0.00	0.61 $\pm$ 0.06	0.93 $\pm$ 0.25	0.66$\pm$0.09
W-COND	0.93$\pm$0.25	0.66$\pm$0.10	0.87 $\pm$ 0.34	0.65 $\pm$ 0.10	0.40 $\pm$ 0.49	0.57 $\pm$ 0.12	0.60 $\pm$ 0.49	0.62 $\pm$ 0.13
<i>Average</i>	<i>0.98</i>	<i>0.65</i>	<i>0.96</i>	<i>0.64</i>	<i>0.80</i>	<i>0.61</i>	<i>0.84</i>	<i>0.66</i>

Table 8: Tool-calling counts for the FULL and W-COND prompt styles: average calls per tool per sample (mean $\pm$ std). Cell shading is a linear interpolation between the smallest and largest average within each prompt style, and encodes how often a tool was called, not how well.

Tool	FULL				W-COND			
	GPT-5-mini	Gemini-3-Flash	Qwen3-4B-Q8	Qwen3.5-4B-Q8	GPT-5-mini	Gemini-3-Flash	Qwen3-4B-Q8	Qwen3.5-4B-Q8
simulate_design	1.0 $\pm$ 0.0	1.0 $\pm$ 0.0	1.0 $\pm$ 0.0	1.0 $\pm$ 0.0	1.0 $\pm$ 0.0	1.2 $\pm$ 0.5	1.0 $\pm$ 0.0	1.0 $\pm$ 0.0
optimize_design	1.0 $\pm$ 0.0	1.0 $\pm$ 0.0	1.0 $\pm$ 0.0	1.0 $\pm$ 0.0	1.0 $\pm$ 0.0	1.0 $\pm$ 0.0	1.0 $\pm$ 0.0	1.0 $\pm$ 0.0
convert_design_to_stl	0.7 $\pm$ 0.5	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	1.2 $\pm$ 0.4	1.0 $\pm$ 0.0	0.8 $\pm$ 0.5	1.0 $\pm$ 0.0
render_design	1.0 $\pm$ 0.0	0.9 $\pm$ 0.2	0.0 $\pm$ 0.0	0.8 $\pm$ 0.5	0.5 $\pm$ 0.5	0.5 $\pm$ 0.5	0.3 $\pm$ 0.4	0.0 $\pm$ 0.0
create_problem	0.4 $\pm$ 0.5	0.0 $\pm$ 0.0	1.0 $\pm$ 0.0	1.0 $\pm$ 0.0	0.1 $\pm$ 0.2	0.0 $\pm$ 0.0	0.6 $\pm$ 0.5	0.0 $\pm$ 0.0
get_problem_details	0.2 $\pm$ 0.4	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0

4.1 Workflow Evaluation

Table 6 reports task completion (TC) and combined overall (CO) scores for the seven Beams2D prompt styles and four LLM backends.² GPT-5-mini and Gemini-3-Flash achieve near-perfect task completion on most styles. Average TC is lower for Qwen3-4B (55%) but increases to 78% for Qwen3.5-4B.

On Beams2D, among the five W-styles, W-COND has the lowest task-completion across the four backends. The highest completion rate is 0.93, obtained by GPT-5-mini. This prompt requires the agent to select an operation using the output of a preceding tool call. Figure 3 shows one example. Gemini-3-Flash selects the correct branch and produces a design that passes parameter validation (IoU = 0.58). Qwen3-4B selects the wrong branch, fails parameter validation, and produces a different topology (IoU = 0.37).

The combined overall score is the hierarchical workflow score S_{workflow} defined in Appendix B. The two proprietary models obtain similar scores, whereas the average CO increases from 0.53 for Qwen3-4B to 0.63 for Qwen3.5-4B. No pairwise model difference remains significant after correction on Beams2D. The scores contain many ties, so this result indicates limited statistical discrimination rather than model equivalence. On Photonics2D, 14 of the 18 model-pair comparisons remain significant after correction. Gemini-3-Flash outperforms the other backends on most styles with large effects (rank-biserial ≥ 0.75), and the improvement from Qwen3-4B to Qwen3.5-4B is significant on the distractor and conditional styles (BH-corrected $q < 0.05$). Appendix D.1 reports the complete tests and effect sizes.

Task completion and CO measure different outcomes. A failed workflow can receive a nonzero CO when the agent still produces a partial design or makes some valid tool calls. For example, Qwen3-4B obtains CO = 0.57 with TC = 0.40 on W-COND. On FULL and NATURAL, it obtains TC = 0 but CO = 0.45 and 0.29, respectively.

These differences follow from the task-completion criteria. On FULL, Qwen3-4B completes the earlier operations but consistently omits the final *render_design* call. The W-style prompts explicitly name each operation, including STL export, and therefore provide more guidance for task sequencing. On NATURAL, Qwen3-4B proceeds with optimization instead of recognizing the missing parameters and calling *ask_human_for_clarification*. Qwen3.5-4B

²A small number of Qwen3.5-4B runs (1–2 per affected style) were lost because of JSON parsing errors, giving $n=13$ –14 instead of 15 for three workflow styles. All other model-style cells contain the full $n=15$.

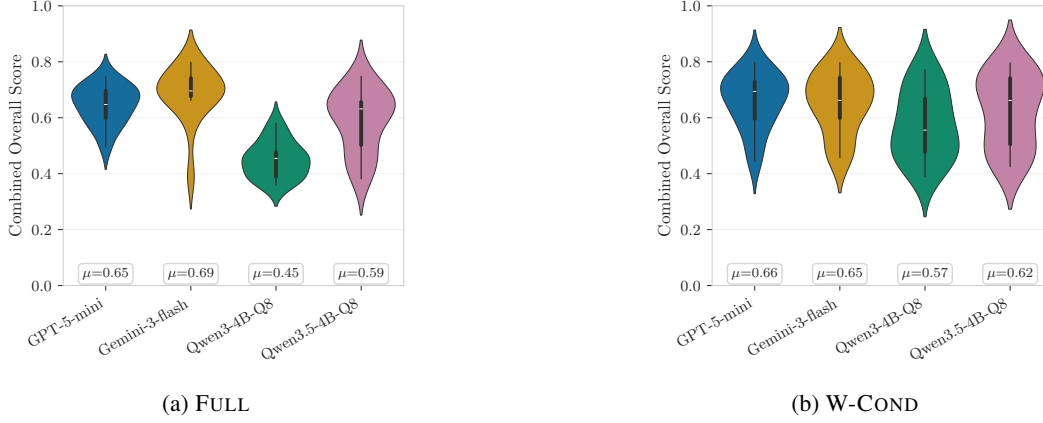


Figure 4: Combined overall score distributions for the FULL (a) and W-COND (b) prompt styles by model backend.

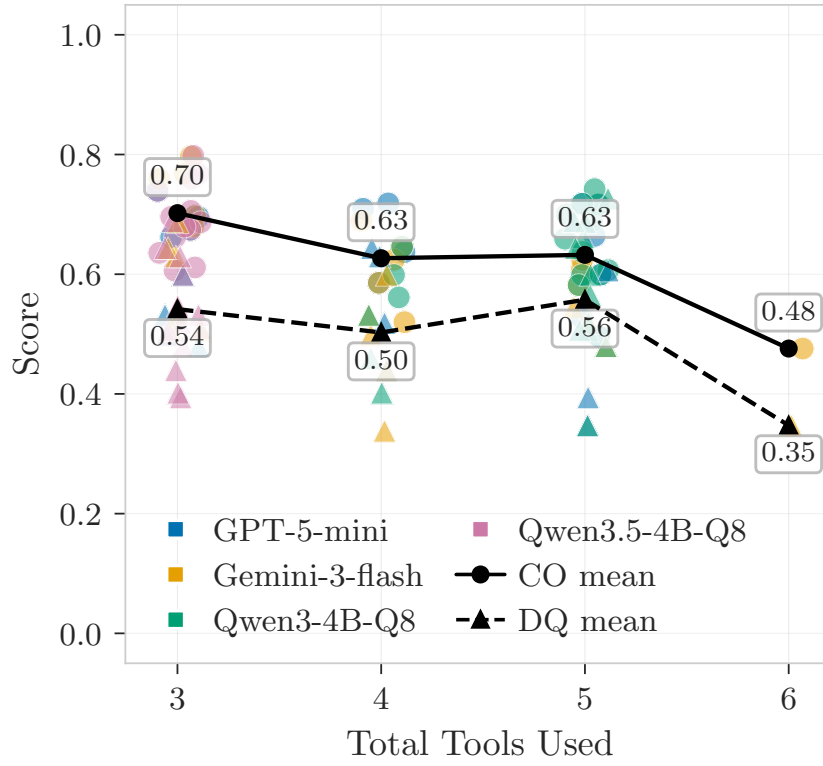


Figure 5: Mean combined overall score (CO, solid line) and design quality score (DQ, dashed line) for the W-RAND style, grouped by the total number of tool calls.

improves completion on FULL to $TC=0.73$, but reaches only $TC=0.33$ on NATURAL. The newer model therefore follows the complete explicit workflow more reliably, while detecting underspecified requests remains difficult.

Table 8 shows the corresponding tool-call counts. On FULL, GPT-5-mini and Gemini-3-Flash call each expected tool once. Qwen3-4B makes redundant calls and omits *render_design*, whereas Qwen3.5-4B follows the expected trace more closely. Figure 4 shows the resulting CO distributions for FULL and W-COND. On W-RAND, additional tool calls reduce CO without improving design quality, which remains near 0.5 (Figure 5). Section 5 discusses this tool-use inefficiency.

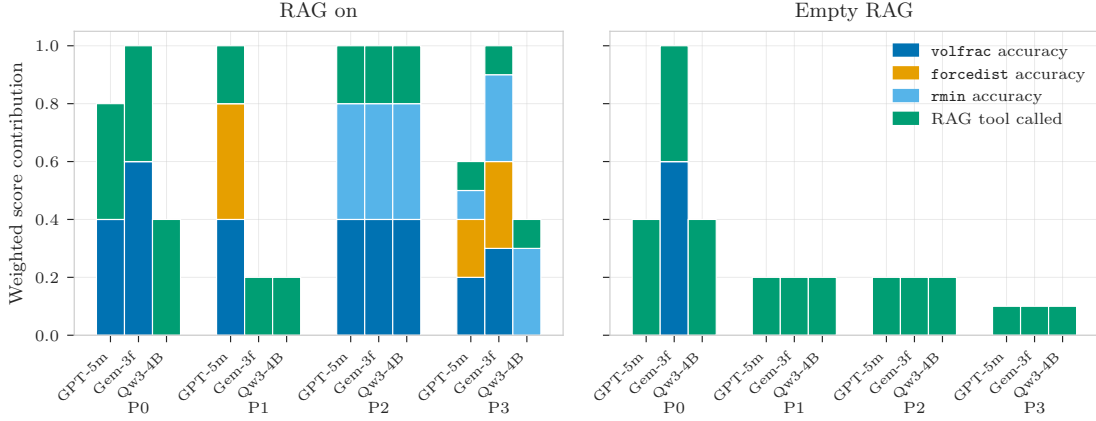


Figure 6: Weighted RAG score contributions by prompt and LLM backend under RAG-on and Empty RAG conditions (three runs per cell). RAG-off is omitted because all scores are zero by construction.

Table 9: Workflow step completion for the EXPLICIT and NATURAL prompt styles: average completion rate per workflow step (mean $\pm$ std over the runs in that cell). Shading is the completion rate.

Step	EXPLICIT		NATURAL	
	GPT-5-mini	Gemini-3-Flash	GPT-5-mini	Gemini-3-Flash
Generate cmd	100% $\pm$ 0.0	100% $\pm$ 0.0	90% $\pm$ 31.6	100% $\pm$ 0.0
Submit job	100% $\pm$ 0.0	100% $\pm$ 0.0	80% $\pm$ 42.2	100% $\pm$ 0.0
Monitor job	90% $\pm$ 31.6	100% $\pm$ 0.0	80% $\pm$ 42.2	100% $\pm$ 0.0
Evaluate	70% $\pm$ 48.3	100% $\pm$ 0.0	50% $\pm$ 52.7	100% $\pm$ 0.0

Photonics2D evaluation. Table 7 reports the results for all four backends on three Photonics2D prompt styles. Several design-quality components provide little discrimination on this problem: background pixels inflate pixel accuracy, the optimizer directly enforces the constraint term, and the multi-component photonic designs receive uniformly low 2D-connectivity scores. We therefore emphasize task completion and tool use.

All four models complete every W-RAND and W-DISTRACT trial ($TC = 1.00$). Performance is substantially lower on W-COND. Gemini-3-Flash achieves the highest completion rate at 53%, followed by Qwen3.5-4B at 47%, GPT-5-mini at 40%, and Qwen3-4B at 20%. For comparison, the highest completion rate on Beams2D W-COND is 93%.

Most Photonics2D W-COND failures result from selecting the wrong conditional branch rather than omitting the export operation. Three models call *convert_design_to_stl* in every trial. Qwen3-4B calls it in 8 of 15 trials, accounting for seven failures caused by a missing export. Among the 29 failed trials that perform the export, 28 select the opposite branch. These trials exchange the branch-specific parameters (*threshold* and *mirror_y*) while retaining the correct shared parameters (*scale_xy* and *scale_z*). The agents therefore usually preserve the parameter values but apply the conditional comparison incorrectly. Appendix D.3 provides the tool-call heatmaps and score distributions. The present experiment does not establish why branch inversion occurs more frequently on Photonics2D.

4.2 RAG Evaluation

Having established workflow performance, we next evaluate whether document retrieval improves engineering parameter selection. Figure 6 reports the weighted score contributions for four prompts and three LLM backends: GPT-5-mini, Gemini-3-Flash, and Qwen3-4B.³ Each model is evaluated three times under the three retrieval conditions defined in Section 3.4.2.

The RAG-off condition receives a score of zero by definition because the gated scoring mechanism awards parameter credit only when the value is supported by retrieved content. We therefore omit this condition from the figure. Scores are generally highest with RAG-on, when the reference documents are indexed, and decrease substantially with Empty

³Qwen3.5-4B is excluded from the RAG and HPC evaluations because it was added after these experiments were completed.

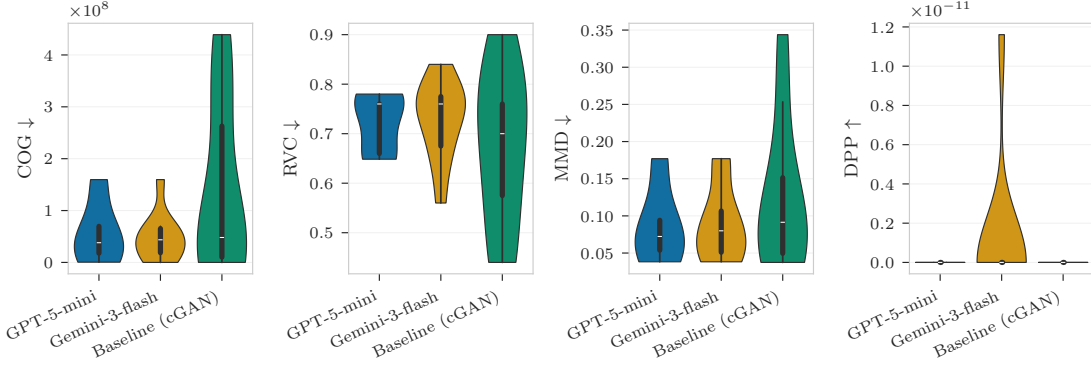


Figure 7: Offline model quality metrics (COG, RVC, MMD, DPP) for agent-trained cGAN models vs. EngiBench baselines. Arrows indicate desired direction. Values averaged across available seeds.

RAG. This comparison shows that providing access to the retrieval tool without the relevant documents does not reproduce the RAG-on results.

Prompt P0 is an exception for Gemini-3-Flash, which returns the expected volume fraction even when the index is empty. The requested value, 0.35, is common in topology optimization, appears in Sigmund’s 99-line benchmark [63], and is also the EngiBench default. The result may therefore reflect prior model knowledge or a common default, although the experiment cannot establish its source. Prompts P2 and P3 instead request parameters from the more recent paper [28]. No model returns the complete expected parameter set for these prompts without the indexed documents.

Performance also varies by model and prompt. Gemini-3-Flash obtains higher scores than GPT-5-mini on P0 and P3, whereas GPT-5-mini scores higher on P1. Qwen3-4B generally obtains lower scores than the proprietary models, except on P2, where all three models receive the same score.

4.3 HPC Training Evaluation

The third evaluation examines whether agents can execute long-running training workflows on remote infrastructure. We evaluate only GPT-5-mini and Gemini-3-Flash because each trial requires substantial wall-clock time for remote submission, training, and monitoring.

Table 9 reports completion of script generation, job submission, monitoring, and model evaluation under two prompt styles. The EXPLICIT prompts enumerate the required tool calls, whereas the NATURAL prompts describe the same workflow without naming each step. Gemini-3-Flash completes all four steps in every tested trial under both prompt styles. GPT-5-mini reaches the final evaluation step in 70% of EXPLICIT trials and 50% of NATURAL trials. Its job-submission and monitoring completion rates also decrease from 90% to 80%.

Inspection of the tool traces shows that GPT-5-mini most often stops before calling *evaluate_model*. The affected runs contain no recorded timeout or tool exception; the agent simply issues no subsequent call. Explicit step enumeration improves completion in these experiments, but the evaluation does not isolate why GPT-5-mini stops during some longer workflows. Gemini-3-Flash completes every tested workflow under both prompt styles.

Figure 7 compares the offline quality metrics of the models produced by completed cGAN runs with the corresponding EngiBench references. The agent-trained models obtain similar metric values in the completed trials, with some variation across runs.

4.4 Training-Code Authoring

We evaluate seven training-code variants on GPT-5-mini and Gemini-3-Flash: T2–T5, the functional variants of T4 and T5, and T6v2. Each variant is evaluated over ten seeds using the prompts reproduced in Appendix C.4. Section 3.4.4 and Figure 2 define the tiers. Table 10 reports the composite score S_{tc} and its model-quality component. Figures 8 and 9 show variation across tiers and compare the canonical and functional evaluators. Appendix D.1 provides the statistical tests and weight-sensitivity analysis.

At T2, the mean model-quality score is 0.74 for GPT-5-mini and 0.81 for Gemini-3-Flash. The difference between the models is larger at T4, where the agent writes the network architecture. Under the canonical evaluator, GPT-5-mini

Table 10: Training-code results (mean $\pm$ std over ten seeds). S_{tc} = composite score; MQ = graded model-quality term ($\min(1, \text{MMD}_{\text{ref}}/\text{MMD}_{\text{agent}})$, per seed). **Bold** = better model per tier per metric.

Tier	GPT-5-mini		Gemini-3-Flash	
	$S_{tc} \uparrow$	MQ $\uparrow$	$S_{tc} \uparrow$	MQ $\uparrow$
T2 (plumbing)	0.85 $\pm$ 0.20	0.74 $\pm$ 0.37	0.89 $\pm$ 0.19	0.81 $\pm$ 0.33
T3 (training loop)	0.82 $\pm$ 0.23	0.68 $\pm$ 0.41	0.85 $\pm$ 0.18	0.74 $\pm$ 0.33
T4 (architecture)	0.55 $\pm$ 0.24	0.21 $\pm$ 0.42	0.77 $\pm$ 0.20	0.59 $\pm$ 0.38
T4-func	0.81 $\pm$ 0.26	0.74 $\pm$ 0.42	0.77 $\pm$ 0.22	0.60 $\pm$ 0.38
T5 (whole module)	0.65 $\pm$ 0.26	0.40 $\pm$ 0.52	0.74 $\pm$ 0.23	0.53 $\pm$ 0.42
T5-func	0.41 $\pm$ 0.02	0.00 $\pm$ 0.00	0.70 $\pm$ 0.32	0.54 $\pm$ 0.43
T6v2 (open synth.)	0.79 $\pm$ 0.21	0.70 $\pm$ 0.37	0.89 $\pm$ 0.14	0.88 $\pm$ 0.25

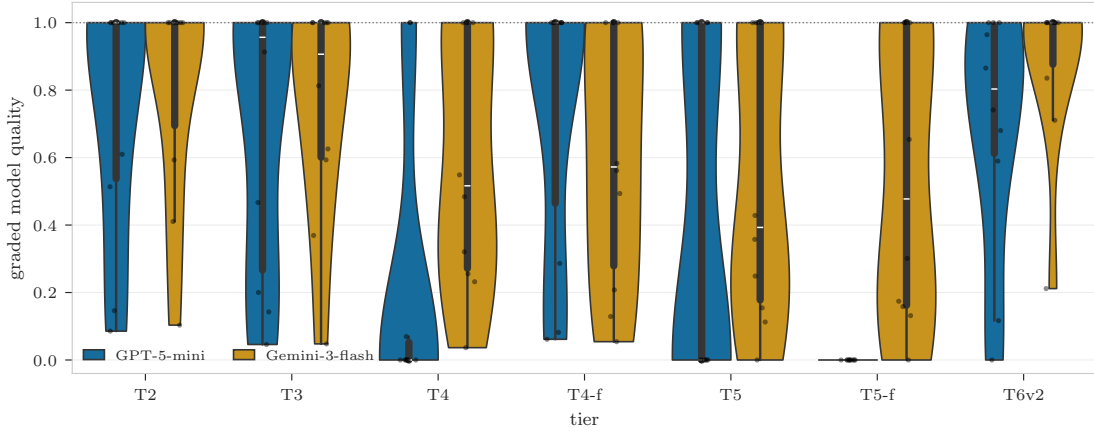


Figure 8: Per-seed model-quality scores by training-code tier and model backend. Violin plots summarize ten seeds; boxes show the median and interquartile range, and points show individual seeds.

scores 0.21 and Gemini-3-Flash scores 0.59. The GPT-5-mini programs complete training, but several checkpoints cannot be loaded into the reference cGAN class because the authored layer names and structure do not match the reference implementation.

The functional evaluator instead loads the class written by the agent. This raises the GPT-5-mini T4 score from 0.21 to 0.74 (Figure 9), showing that much of the canonical penalty results from interface incompatibility rather than training failure. The functional T5 evaluation adds another requirement: the agent must supply a working sampling function. GPT-5-mini scores 0.40 under the canonical evaluator, but its authored sampler produces nearly constant outputs that do not respond to conditioning and therefore receives zero model-quality credit under the functional evaluator. Gemini-3-Flash scores 0.53 and 0.54 on the canonical and functional T5 evaluations, respectively.

At T6v2, both models select a conditional variational autoencoder rather than the reference cGAN. Gemini-3-Flash implements a convolutional encoder and decoder with a reconstruction-plus-KL objective. GPT-5-mini implements a fully connected model with feature-wise linear modulation conditioning. Both programs complete training and provide a working sampling function. Figure 10 shows designs produced by the authored generators.

GPT-5-mini obtains a mean model-quality score of 0.70, while Gemini-3-Flash obtains 0.88. Gemini-3-Flash has a mean MMD of 0.094, compared with 0.19 for the seed-matched reference cGAN, and obtains lower MMD on seven of ten seeds. This comparison is specific to these trials and does not establish general superiority of the VAE architecture.

Performance is not monotonic across tiers: model-quality scores decrease at T4 and T5 and increase again at T6v2 (Table 10; Appendix D.1). Some lower scores result from interface incompatibility or sampling-function failure rather than unsuccessful training. The tiers should therefore be interpreted as distinct authoring tasks.

Photonics2D replication. We repeat T2, T4, and T6v2 on Photonics2D using the same two model backends and ten seeds per tier. Each trial authors code specifically for its assigned problem. Table 11 reports the results, while Appendix D.3.3 separately tests whether T6v2 programs can run unchanged on the other problem.

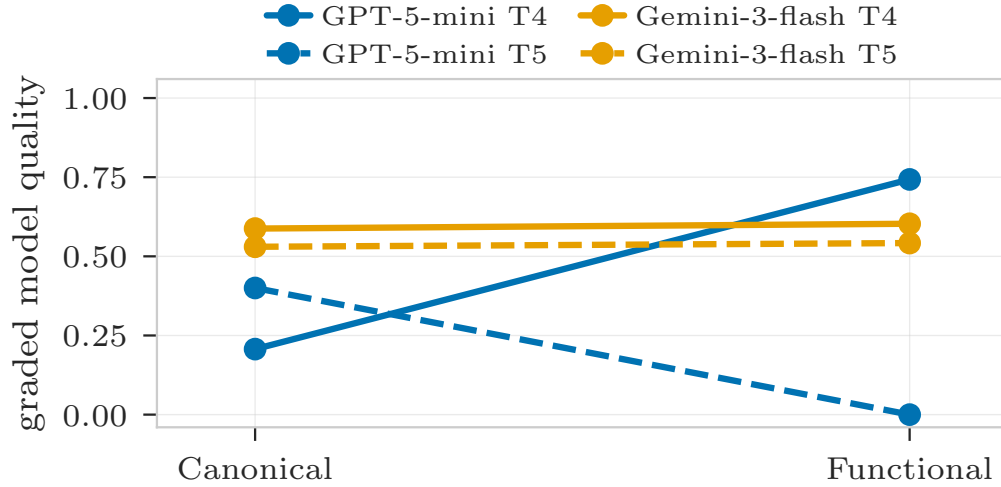


Figure 9: Mean model-quality scores at T4 and T5 under the canonical and functional evaluators. The canonical evaluator loads the checkpoint using the reference cGAN class; the functional evaluator uses the agent-authored model class and sampling function.

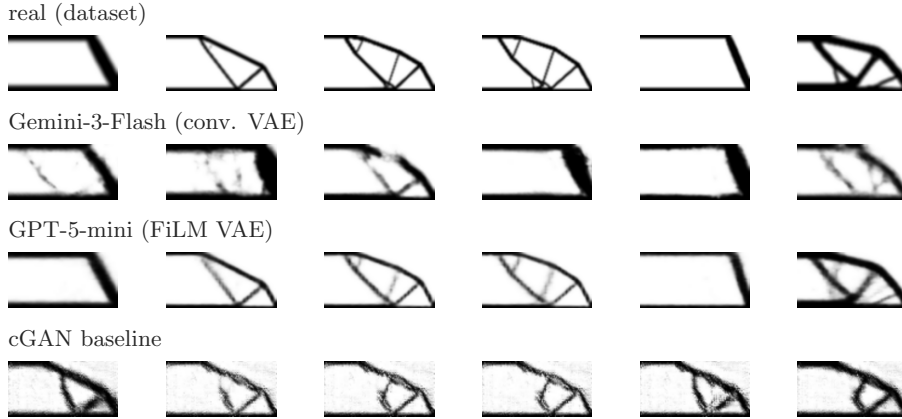


Figure 10: Beams2D designs produced by the open-synthesis generators for seed 5. Columns correspond to fixed load conditions taken from dataset designs. Rows show the dataset design, the Gemini-3-Flash convolutional VAE, the GPT-5-mini fully connected VAE with FiLM conditioning, and the reference cGAN. Designs are shown without thresholding.

At T2, the mean model-quality scores are 0.94 for GPT-5-mini and 0.97 for Gemini-3-Flash. At T4, the canonical evaluator successfully grades three GPT-5-mini checkpoints and six Gemini-3-Flash checkpoints. The remaining checkpoints cannot be loaded into the reference class. Among the successfully graded checkpoints, mean MMD ranges from 0.54 to 0.64, compared with 0.86 for the reference cGAN. The low aggregate T4 scores therefore primarily reflect reference-interface incompatibility.

At T6v2, mean MMD is 0.45 for GPT-5-mini and 0.22 for Gemini-3-Flash. Their corresponding model-quality scores are 0.90 and 1.00. One GPT-5-mini sampler is rejected by the degeneracy gate. Appendix Figure 15 shows designs from both authored generators and the reference cGAN.

These results require two qualifications. First, the reference cGAN has an MMD of 0.86 on Photonics2D, so exceeding this reference does not by itself establish strong absolute model performance. Second, the Page trend test [51] across the three selected tiers is not significant ($p=0.11$). The decrease at T4 followed by an increase at T6v2 is also non-

Table 11: Training-code results on Photonics2D (mean $\pm$ std over ten seeds). S_{tc} = composite score; MQ = graded model-quality term ($\min(1, \text{MMD}_{\text{ref}}/\text{MMD}_{\text{agent}})$), per seed). **Bold** = better model per tier per metric. Gray text shows Beams2D results on the same tiers for comparison.

Tier	GPT-5-mini		Gemini-3-Flash	
	$S_{tc} \uparrow$	MQ $\uparrow$	$S_{tc} \uparrow$	MQ $\uparrow$
T2 (plumbing)	0.96 $\pm$ 0.05	0.94 $\pm$ 0.08	0.97 $\pm$ 0.04	0.97 $\pm$ 0.07
T4 (architecture)	0.59 $\pm$ 0.27	0.30 $\pm$ 0.48	0.78 $\pm$ 0.29	0.60 $\pm$ 0.52
T6v2 (open synth.)	0.89 $\pm$ 0.17	0.90 $\pm$ 0.32	0.95 $\pm$ 0.02	1.00 $\pm$ 0.01
<i>Beams2D (same tiers, for comparison)</i>				
T2 (plumbing)	0.85 $\pm$ 0.20	0.74 $\pm$ 0.37	0.89 $\pm$ 0.19	0.81 $\pm$ 0.33
T4 (architecture)	0.55 $\pm$ 0.24	0.21 $\pm$ 0.42	0.77 $\pm$ 0.20	0.59 $\pm$ 0.38
T6v2 (open synth.)	0.79 $\pm$ 0.21	0.70 $\pm$ 0.37	0.89 $\pm$ 0.14	0.88 $\pm$ 0.25

monotonic. The Photonics2D experiment therefore provides a qualitative cross-domain replication, not evidence of a general trend across authoring tiers (Appendix D.1).

5 Discussion

5.1 Key Findings

We organize the findings around the four benchmark dimensions introduced in Section 1: workflow execution, retrieval-assisted parameter selection, HPC orchestration, and training-code authoring.

Workflow execution. GPT-5-mini and Gemini-3-Flash achieve 96–97% average task completion across the seven Beams2D prompt styles (Table 6). At the same 4B parameter count, task completion increases from 55% for Qwen3-4B to 78% for Qwen3.5-4B. The largest improvements occur on FULL (0.00 $\rightarrow$ 0.73) and W-DERIVED (0.47 $\rightarrow$ 0.85). The difference also appears on Photonics2D W-COND, where completion increases from 0.20 to 0.47, although the highest rate among all models is only 0.53 (Table 7). Conditional branching and the detection of underspecified requests therefore remain less reliable than explicitly specified workflows.

Tool-call efficiency provides information beyond task completion. On W-RAND, increasing the number of calls from three to four or five reduces the combined score from 0.70 to 0.63 without improving design quality, which remains near 0.5 (Figure 5). Additional calls therefore impose a score penalty without improving the resulting design in these trials.

Retrieval-assisted parameter selection. Scores are generally highest when the reference documents are indexed and decrease substantially when the retrieval index is empty (Figure 6). The RAG-off condition receives zero parameter credit by construction because the gated scorer requires retrieved evidence. The comparison between RAG-on and Empty RAG provides the relevant evidence that indexed document content improves parameter selection.

HPC orchestration. Among the two proprietary models tested, Gemini-3-Flash completes all four stages of the supplied training workflow in every trial. GPT-5-mini reaches the final evaluation stage in 70% of trials with explicit prompts and 50% with natural prompts (Table 9). Its incomplete traces most often stop before the final *evaluate_model* call, without a recorded timeout or tool exception. Thus, both models can submit and monitor remote jobs, but completion of the full workflow depends on the model and prompt format.

Training-code authoring. At T2, mean model-quality scores are 0.74 for GPT-5-mini and 0.81 for Gemini-3-Flash (Table 10). Results become more model- and tier-dependent when the agent writes the architecture or sampling function. At T4, GPT-5-mini scores 0.21 under the canonical evaluator but 0.74 when the functional evaluator loads the agent’s own class (Figure 9). This difference identifies reference-interface incompatibility rather than unsuccessful training. At T5-func, the GPT-5-mini sampling function ignores conditioning and receives zero model-quality credit, whereas Gemini-3-Flash scores 0.54.

At the open-synthesis tier, both models implement conditional VAEs rather than the reference cGAN. Gemini-3-Flash obtains a mean MMD of 0.094, compared with 0.19 for the seed-matched reference, and has lower MMD on seven of ten Beams2D seeds (Figure 10). These experiments show that the functional evaluator can assess an agent-authored model from a different generative family. They also show that checkpoint compatibility, sampling behavior, and model quality are separate sources of success or failure. These conclusions are limited to the tested models, prompts, seeds, and engineering problems.

5.2 Comparison to Prior Work

Table 1 summarizes the capabilities covered by related systems and benchmarks. The following discussion positions our results against the closest work for each of the four evaluation dimensions.

Workflow Performance The closest multi-agent engineering system is the 9-agent MAS with Design State Graph (DSG) coordination [40]. Direct numerical comparison is not meaningful because the two systems address structurally different tasks: DSG-MAS targets full design-state management across coupled requirement–design–test loops, whereas our benchmark evaluates tool-calling workflows on structured engineering pipelines. We therefore position DSG-MAS as a complementary contribution rather than a numerical baseline; matched-task comparison would require a shared evaluation framework that does not currently exist.

Our workflow benchmark extends existing engineering evaluations by testing multi-step tool orchestration. FDM-Bench [18] evaluates LLMs on G-code anomaly detection and user queries but does not evaluate connected tool workflows. General-purpose benchmarks such as ToolSandbox [38] and ACEBench [8] report performance differences between open-source and proprietary models. We observe a similar difference between the proprietary models and Qwen3-4B, although Qwen3.5-4B improves substantially over Qwen3-4B at the same parameter count. For multi-turn professional tasks, τ -bench [75] reports task success below 50% and pass⁸ below 25%. These values are not directly comparable with ours because the benchmarks use different tasks, tools, and success criteria.

RAG Evaluation While RAG is a well-established technique [35, 22], its evaluation in engineering contexts has focused on document comprehension rather than actionable parameter extraction. Doris *et al.* [16] benchmark multimodal LLM understanding of engineering documentation (rule comprehension, compliance, and extraction), but do not evaluate whether retrieved information is correctly applied to downstream tool calls. Our evaluation addresses this gap by testing whether retrieved parameters are used in downstream tool calls. The comparison between populated and empty indexes holds retrieval-tool access constant while varying the indexed content. For P2 and P3, which require parameter sets from the reference papers, no model returned the complete expected set when those documents were absent from the index.

HPC Orchestration Among the studies reviewed, ALL-FEM [15] is the closest work on automated simulation execution. It uses multiple LLM agents to generate and execute FEniCS code in Google Colab and reports 71.79% accuracy across 39 FEA benchmarks. ALL-FEM evaluates simulation-code generation and execution in a notebook environment. Our HPC evaluation instead records whether an agent generates a SLURM script, submits and monitors a remote training job, and evaluates the resulting model. Figure 7 reports the model-quality metrics from completed runs alongside the corresponding EngiBench references.

Training-Code Authoring MLGym, MAgentBench, and MLE-bench evaluate agents that write or modify machine-learning training code [44, 29, 5]. Their reported success measures are not directly comparable with ours because they use standard machine-learning datasets or competition scores, whereas our functional evaluator computes MMD from generated engineering designs. The closest qualitative comparison is MLGym’s finding that agents more often tune existing programs than implement a different model architecture.

Our T6v2 experiment removes the reference architecture and supplies only the required data and evaluation interfaces. Both agents select established conditional VAE architectures rather than the reference cGAN and complete the SLURM training workflow. Gemini-3-Flash implements a convolutional VAE and obtains a model-quality score of 0.88. Its mean MMD is 0.094, compared with 0.19 for the seed-matched reference cGAN, and it obtains lower MMD on seven of ten seeds. GPT-5-mini implements a fully connected VAE, obtains a model-quality score of 0.70, and does not improve on the reference on average. These results show that, in the tested setting, an agent can implement and train a generative model family different from the supplied reference. They do not establish an algorithm that is novel to the research literature. Unlike the benchmarks above, our evaluation additionally requires the authored program to complete a remote HPC/SLURM workflow.

General-Purpose Coding Agents General-purpose coding agents—such as OpenAI Codex [48], Claude Code [3], and OpenClaw [50]—can write and execute arbitrary code and may call the EngiBench and EngiOpt APIs without pre-bound tool abstractions. The task definitions and scoring criteria do not conceptually depend on ENGIAI’s seven-agent decomposition. The current software evaluates ENGIAI directly; evaluating another system requires an adapter that maps its tool traces and artifacts to the schema expected by the scorers.

5.3 Implications for Engineering-Agent Development

Results in Table 3 support five recommendations for developing and evaluating engineering agents. First, workflow execution, retrieval-assisted parameter selection, remote-job orchestration, and code authoring should be reported separately because a single end-to-end success rate hides distinct failure mechanisms. Second, evaluation should assess both the execution trace and the resulting engineering artifact because valid tool calls can still use the wrong branch, parameter, or trained model. Third, prompts should test ambiguity, competing plausible values, derived quantities, conditional decisions, and retention of multiple parameter sets rather than only explicit step-by-step instructions. Fourth, retrieval effectiveness should be assessed by comparing populated and empty indexes while holding retrieval-tool access constant. A no-retrieval condition may be reported separately to verify agent and scorer behavior, but it should not be treated as evidence of retrieval effectiveness when the scorer requires a retrieval call. Fifth, authored training programs should be evaluated functionally using the agent’s own model and sampling interfaces, while compatibility with a reference implementation should be reported separately.

ENGIAI is intended as a reference implementation and evaluation testbed, not as a required agent architecture. Researchers can adapt the task definitions and scorers to another supervisor, a single agent, or a coding agent by providing compatible tool, trace, and artifact interfaces. Engineering teams can adapt these tests as acceptance criteria for grounding, tool use, long-running execution, and artifact validity before deploying an agent on a new workflow. The present experiments do not establish readiness for unsupervised production use.

5.4 Limitations and Threats to Validity

The evaluation covers two EngiBench problems, Beams2D and Photonics2D, and therefore does not represent the full range of engineering problems available in EngiBench. We also did not conduct a study with practicing engineers. The experiments consequently do not assess interactive usability, the quality of human interventions, or how effectively users can redirect an agent after a failure.

Only four LLM backends were evaluated, and the RAG, HPC, and training-code evaluations use subsets of these models. The HPC benchmark is further limited to two generative-model families (cGAN and diffusion), two proprietary LLM backends (GPT-5-mini and Gemini-3-Flash), one cluster, and one training setup. The corresponding orchestration results should therefore be interpreted within these experimental conditions. Finally, we do not include a matched single-agent baseline with identical tool access. The experiments therefore do not isolate the effect of the multi-agent supervisor decomposition.

Appendix D.1 reports bootstrap confidence intervals and the applicable non-parametric tests for all four evaluations. It also tests the sensitivity of the training-code ranking to the composite weights. We did not perform corresponding weight-sensitivity analyses for the other composite scores, and we did not vary temperature or prompt wording systematically.

The training-code experiments cover one reference model family, two engineering problems, two proprietary LLM backends, and one prompt wording. The Photonics2D results provide a second problem instance, but they do not establish generalization to other engineering domains or generative-model families.

Model quality is compared with one reference training run for each seed. Reference cGAN MMD ranges from approximately 0.03 to 0.59 across the ten Beams2D seeds. An authored model can therefore outperform a weak reference run without outperforming the reference method in general. Repeated reference runs would support a stronger comparison.

T2–T5 retain parts of the public EngiOpt implementation. An agent could reproduce these regions from prior exposure rather than derive them during the trial. T6v2 reduces this concern by withholding the reference architecture, but the selected VAE architectures are established methods. We therefore interpret T6v2 as evidence that the agent can implement a model not constrained to the supplied cGAN, not as evidence of a new generative algorithm.

API responses varied slightly across repeated runs despite a sampling temperature of zero. Tables 6 and 7 report the resulting standard deviations.

6 Conclusion and Future Work

We evaluated four capabilities of LLM agents in engineering design workflows: tool execution, retrieval-assisted parameter selection, remote training-job orchestration, and training-code authoring. ENGIAI provides the multi-agent, tool-connected implementation used in these experiments.

The experiments yield four main findings. First, GPT-5-mini and Gemini-3-Flash complete 96–97% of the structured Beams2D workflows, compared with 55% for Qwen3-4B and 78% for Qwen3.5-4B. Second, indexed documents improve numerical parameter selection relative to an empty retrieval index. Third, Gemini-3-Flash completes every tested HPC pipeline, whereas GPT-5-mini reaches the final evaluation step in 70% of explicit-prompt trials and 50% of natural-prompt trials. Fourth, successful orchestration of a supplied training program does not imply uniform success when the agent must write that program. The authoring benchmark identifies separate failures in checkpoint compatibility, sampling behavior, and model quality. At T6v2, both proprietary models implement conditional VAEs rather than the supplied cGAN; Gemini-3-Flash obtains lower MMD than the seed-matched Beams2D reference in seven of ten trials.

The study covers two engineering problems and four LLM backends, but the RAG, HPC, and authoring evaluations use subsets of those backends. The retrieval index contains only two papers, the authoring benchmark uses two proprietary models and one prompt formulation, and model quality is compared with a single reference run per seed. The results should therefore not be generalized beyond the tested models, prompts, and problems.

Future work should evaluate additional engineering problems, model backends, prompt formulations, and sampling temperatures. Retrieval should be tested with larger and noisier document collections. The authoring benchmark should include other generative-model families, including diffusion models, and compare authored models against distributions of repeated reference runs. A further extension would raise the authoring target from the training program to the simulator itself, supplying a multi-page problem specification as input and scoring the generated simulator against reference solutions and physical validity checks. A matched single-agent baseline is also needed to determine whether the multi-agent decomposition itself improves performance. Finally, experiments with larger tool sets exposed through tool APIs or MCP would test how workflow reliability changes as the system expands.

The principal implication is that engineering agents should not be assessed from successful demonstrations alone. Reliable deployment requires capability-specific tests, controlled information conditions, inspection of intermediate artifacts, and validation of the resulting engineering artifact. ENGIAI provides one implementation of this evaluation approach, while the four evaluation dimensions define a basis for future comparisons among systems that expose compatible evidence.

Conflict of Interest

There are no conflicts of interest.

Data Availability Statement

The code of this project is publicly available on GitHub.

References

- [1] Deepak Bhaskar Acharya, Karthigeyan Kuppan, and B. Divya. Agentic AI: Autonomous intelligence for complex goals—a comprehensive survey. *IEEE Access*, 13:18912–18936, 2025. doi:10.1109/ACCESS.2025.3532853. URL <https://doi.org/10.1109/ACCESS.2025.3532853>.
- [2] Erik Andreassen, Anders Clausen, Mattias Schevenels, et al. Efficient topology optimization in MATLAB using 88 lines of code. *Structural and Multidisciplinary Optimization*, 43(1):1–16, January 2011. ISSN 1615-147X, 1615-1488. doi:10.1007/s00158-010-0594-7. URL <http://link.springer.com/10.1007/s00158-010-0594-7>.
- [3] Anthropic. Claude code, 2025. URL <https://docs.anthropic.com/en/docs/claude-code>.
- [4] Anthropic. Model Context Protocol, 2025. URL <https://modelcontextprotocol.io/specification/2025-11-25>.
- [5] Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, Aleksander Madry, and Lilian Weng. MLE-bench: Evaluating machine learning agents on machine learning engineering. In *The Thirtieth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=6s5uXNWGIh>.
- [6] Achuth Chandrasekhar, Jonathan Chan, Francis Ogoke, et al. Amgpt: A large language model for contextual querying in additive manufacturing. *Additive Manufacturing Letters*, 11:100232, 2024. ISSN 2772-3690. doi:10.1016/j.addlet.2024.100232. URL <https://www.sciencedirect.com/science/article/pii/S2772369024000409>.

- [7] Harrison Chase. LangChain, October 2022. URL <https://github.com/langchain-ai/langchain>.
- [8] Chen Chen, Xinlong Hao, Weiwen Liu, Xu Huang, Xingshan Zeng, Shuai Yu, Dexun Li, Yuefeng Huang, Xiangcheng Liu, Wang Xinzhi, and Wu Liu. ACEBench: A comprehensive evaluation of LLM tool usage. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 12970–12998, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-335-7. doi:10.18653/v1/2025.findings-emnlp.697. URL <https://aclanthology.org/2025.findings-emnlp.697/>.
- [9] Pei Chen, Yichen Cai, Zihong Zhou, et al. An llm-based multi-agent system to assist early-stage product design and evaluation. *Journal of Engineering Design*, 37(3):945–980, 2026. doi:10.1080/09544828.2026.2616583. URL <https://doi.org/10.1080/09544828.2026.2616583>.
- [10] Wei Chen and Faez Ahmed. PaDGAN: Learning to generate high-quality novel designs. *Journal of Mechanical Design*, 143(3):031703, March 2021. doi:10.1115/1.4048626. URL <https://doi.org/10.1115/1.4048626>.
- [11] Wei (Wayne) Chen, Doksoo Lee, Oluwaseyi Balogun, and Wei Chen. GAN-DUF: Hierarchical deep generative models for design under free-form geometric uncertainty. *Journal of Mechanical Design*, 145(1):011703, 2023. doi:10.1115/1.4055898. URL <https://doi.org/10.1115/1.4055898>.
- [12] Ziru Chen, Shijie Chen, Yuting Ning, et al. Scienceagentbench: Toward rigorous assessment of language agents for data-driven scientific discovery. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=6z4YKr0GK6>.
- [13] CrewAI. CrewAI: Framework for orchestrating role-playing autonomous AI agents, 2024. URL <https://github.com/crewAIInc/crewAI>. Open-source Python library.
- [14] Xiang Deng, Yu Gu, Boyuan Zheng, et al. Mind2Web: Towards a Generalist Agent for the Web, December 2023. URL <http://arxiv.org/abs/2306.06070>. arXiv:2306.06070 [cs].
- [15] Rushikesh Deotale, Adithya Srinivasan, Yuan Tian, et al. ALL-FEM: Agentic Large Language models Fine-tuned for Finite Element Methods. *SSRN Electronic Journal*, 2026. doi:10.2139/ssrn.6103826. URL <https://ssrn.com/abstract=6103826>.
- [16] Anna C. Doris, Daniele Grandi, Ryan Tomich, et al. DesignQA: A multimodal benchmark for evaluating large language models’ understanding of engineering documentation. *Journal of Computing and Information Science in Engineering*, 25(2):021009, 2025. doi:10.1115/1.4067333. URL <https://doi.org/10.1115/1.4067333>.
- [17] Mohamed Elrefaie, Janet Qian, Raina Wu, et al. Ai agents in engineering design: A multi-agent framework for aesthetic and aerodynamic car design. In *Volume 3B: 51st Design Automation Conference (DAC)*, IDETC-CIE2025. American Society of Mechanical Engineers, August 2025. doi:10.1115/detc2025-169682. URL <http://dx.doi.org/10.1115/DETC2025-169682>.
- [18] Ahmadreza Eslaminia, Adrian Jackson, Beitong Tian, et al. Fdm-bench: a domain-specific benchmark for evaluating large language models in additive manufacturing. *Manufacturing Letters*, 44:1415–1424, 2025. ISSN 2213-8463. doi:10.1016/j.mfglet.2025.06.161. URL <https://www.sciencedirect.com/science/article/pii/S2213846325001968>. 53rd SME North American Manufacturing Research Conference (NAMRC 53).
- [19] Florian Felten, Gabriel Apaza, Gerhard Bräunlich, et al. Engibench: A framework for data-driven engineering design research. In *Advances in Neural Information Processing Systems 38*, pages 70–113. Neural Information Processing Systems Foundation, 2025. doi:10.52202/085713-0003. URL <https://openreview.net/forum?id=YowD33Q89V>.
- [20] Thibault Formal, Benjamin Piwowarski, and Stéphane Clinchant. Splade: Sparse lexical and expansion model for first stage ranking, 2021. URL <https://arxiv.org/abs/2107.05720>.
- [21] Milton Friedman. The use of ranks to avoid the assumption of normality implicit in the analysis of variance. *Journal of the American Statistical Association*, 32(200):675–701, 1937. doi:10.1080/01621459.1937.10503522. URL <https://doi.org/10.1080/01621459.1937.10503522>.
- [22] Yunfan Gao, Yun Xiong, Xinyu Gao, et al. Retrieval-augmented generation for large language models: A survey, December 2023. URL <https://arxiv.org/abs/2312.10997>. arXiv:2312.10997.
- [23] Juraj Gottweis, Wei-Hung Weng, Alexander Daryin, et al. Accelerating scientific discovery with Co-Scientist. *Nature*, 655:487–496, 2026. doi:10.1038/s41586-026-10644-y. URL <https://doi.org/10.1038/s41586-026-10644-y>.
- [24] Mourad Gridach, Jay Nanavati, Christina Mack, et al. Agentic AI for scientific discovery: A survey of progress, challenges, and future directions. In *Towards Agentic AI for Science: Hypothesis Generation, Comprehension, Quantification, and Validation*, 2025. URL <https://openreview.net/forum?id=TyCYakX9BD>.

- [25] Xingang Guo, Yaxin Li, XiangYi Kong, et al. Toward Engineering AGI: Benchmarking the engineering design capabilities of LLMs. In *Advances in Neural Information Processing Systems* 38, pages 78582–78642. Neural Information Processing Systems Foundation, 2025. doi:10.52202/085713-2372. URL <https://openreview.net/forum?id=Wmsnx7EPe1>.
- [26] George A. Hazelrigg. A framework for decision-based engineering design. *Journal of Mechanical Design*, 120(4): 653–658, December 1998. doi:10.1115/1.2829328. URL <https://doi.org/10.1115/1.2829328>.
- [27] Hongliang He, Wenlin Yao, Kaixin Ma, et al. WebVoyager: Building an end-to-end web agent with large multimodal models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6864–6890. Association for Computational Linguistics, 2024. doi:10.18653/v1/2024.acl-long.371. URL <https://aclanthology.org/2024.acl-long.371/>.
- [28] Liang He, Huayi Wei, and Tian Tian. SOPTX: A High-Performance Multi-Backend Framework for Topology Optimization, May 2025. URL <http://arxiv.org/abs/2505.02438>. arXiv:2505.02438 [math].
- [29] Qian Huang, Jian Vora, Percy Liang, and Jure Leskovec. MAgentBench: Evaluating language agents on machine learning experimentation. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 20271–20309. PMLR, 2024. URL <https://proceedings.mlr.press/v235/huang24y.html>.
- [30] Yayati Jadhav, Peter Pak, and Amir Barati Farimani. Llm-3d print: Large language models to monitor and control 3d printing. *Additive Manufacturing*, 114:105027, 2025. ISSN 2214-8604. doi:10.1016/j.addma.2025.105027. URL <https://www.sciencedirect.com/science/article/pii/S2214860425003926>.
- [31] Shuo Jiang, Min Xie, Frank Youhua Chen, et al. Intelligent design 4.0: Paradigm evolution toward the agentic artificial intelligence era. *Journal of Computing and Information Science in Engineering*, 25(12):120808, 12 2025. ISSN 1530-9827. doi:10.1115/1.4070438. URL <https://doi.org/10.1115/1.4070438>.
- [32] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=VTF8yNQm66>.
- [33] Kiarash Naghavi Khanghah, Zhiling Chen, Lela Romeo, et al. Zero-shot anomaly detection in laser powder bed fusion using multimodal retrieval-augmented generation and large language models. *Journal of Mechanical Design*, 148(7):072001, 2026. doi:10.1115/1.4070585. URL <https://doi.org/10.1115/1.4070585>.
- [34] LangChain, Inc. LangGraph: Build resilient language agents as graphs, 2024. URL <https://github.com/langchain-ai/langgraph>. Open-source Python library.
- [35] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474. Curran Associates, Inc., 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>.
- [36] Xiao Liu, Hao Yu, Hanchen Zhang, et al. AgentBench: Evaluating LLMs as Agents. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=zAdUB0aCTQ>.
- [37] Chris Lu, Cong Lu, Robert Tjarko Lange, et al. The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery, September 2024. URL <http://arxiv.org/abs/2408.06292>. arXiv:2408.06292 [cs].
- [38] Jiarui Lu, Thomas Holleis, Yizhe Zhang, et al. ToolSandbox: A Stateful, Conversational, Interactive Evaluation Benchmark for LLM Tool Use Capabilities. In Luis Chiruzzo, Alan Ritter, and Lu Wang, editors, *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 1160–1183, Albuquerque, New Mexico, April 2025. Association for Computational Linguistics. ISBN 979-8-89176-195-7. doi:10.18653/v1/2025.findings-naacl.65. URL <https://aclanthology.org/2025.findings-naacl.65/>.
- [39] Chang Ma, Junlei Zhang, Zhihao Zhu, et al. AgentBoard: An Analytical Evaluation Board of Multi-turn LLM Agents, December 2024. URL <http://arxiv.org/abs/2401.13178>. arXiv:2401.13178 [cs].
- [40] Soheyl Massoudi and Mark Fuge. Agentic large language models for conceptual systems engineering and design. *Journal of Mechanical Design*, 148(5):051405, 01 2026. ISSN 1050-0472. doi:10.1115/1.4070328. URL <https://doi.org/10.1115/1.4070328>.
- [41] François Mazé and Faez Ahmed. Diffusion models beat GANs on topology optimization. *Proceedings of the AAAI Conference on Artificial Intelligence*, 37(8):9108–9116, 2023. doi:10.1609/aaai.v37i8.26093. URL <https://ojs.aaai.org/index.php/AAAI/article/view/26093>.
- [42] Mehdi Mirza and Simon Osindero. Conditional generative adversarial nets, 2014. URL <https://arxiv.org/abs/1411.1784>.

- [43] Mahmoud Mohammadi, Yipeng Li, Jane Lo, et al. Evaluation and benchmarking of llm agents: A survey. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2*, KDD '25, page 6129–6139, New York, NY, USA, 2025. Association for Computing Machinery. ISBN 9798400714542. doi:10.1145/3711896.3736570. URL <https://doi.org/10.1145/3711896.3736570>.
- [44] Deepak Nathani, Lovish Madaan, Nicholas Roberts, Nikolay Bashlykov, Ajay Menon, Vincent Moens, Amar Budhiraja, Despoina Magka, Vladislav Vorotilov, Gaurav Chaurasia, Dieuwke Hupkes, Ricardo Silveira Cabral, Tatiana Shavrina, Jakob Foerster, Yoram Bachrach, William Yang Wang, and Roberta Raileanu. Mlgym: A new framework and benchmark for advancing ai research agents, 2025. URL <https://arxiv.org/abs/2502.14499>.
- [45] Bo Ni and Markus J. Buehler. Mechagents: Large language model multi-agent collaborations can solve mechanics problems, generate new data, and integrate knowledge. *Extreme Mechanics Letters*, 67:102131, 2024. ISSN 2352-4316. doi:10.1016/j.eml.2024.102131. URL <https://www.sciencedirect.com/science/article/pii/S2352431624000117>.
- [46] Zhenguo Nie, Tong Lin, Haoliang Jiang, and Levent Burak Kara. Topologygan: Topology optimization using generative adversarial networks based on physical fields over the initial domain. *Journal of Mechanical Design*, 143(3):031715, 02 2021. ISSN 1050-0472. doi:10.1115/1.4049533. URL <https://doi.org/10.1115/1.4049533>.
- [47] OpenAI. OpenAI Agents SDK, 2025. URL <https://github.com/openai/openai-agents-python>. Open-source Python library.
- [48] OpenAI. Codex, 2025. URL <https://openai.com/codex/>.
- [49] OpenAI. GPT-5 system card, August 2025. URL <https://openai.com/index/gpt-5-system-card/>. Published August 7, 2025.
- [50] OpenClaw Contributors. OpenClaw, 2025. URL <https://github.com/openclaw/openclaw>. Open-source personal AI assistant.
- [51] Ellis Batten Page. Ordered hypotheses for multiple treatments: A significance test for linear ranks. *Journal of the American Statistical Association*, 58(301):216–230, 1963. doi:10.1080/01621459.1963.10500843. URL <https://doi.org/10.1080/01621459.1963.10500843>.
- [52] Gerhard Pahl, Wolfgang Beitz, Jörg Feldhusen, and Karl-Heinrich Grote. *Engineering Design: A Systematic Approach*. Springer London, London, 3 edition, 2007. ISBN 978-1-84628-318-5. doi:10.1007/978-1-84628-319-2. URL <https://doi.org/10.1007/978-1-84628-319-2>.
- [53] Hyunjee Park and Hayoung Chung. Self-refining topology optimization via an llm-based multi-agent framework, 2026. URL <https://arxiv.org/abs/2605.23273>.
- [54] Shishir G Patil, Huanzhi Mao, Fanjia Yan, et al. The berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=2GmDdhBdDk>.
- [55] Alejandro Pradas-Gómez, Arindam Brahma, and Ola Isaksson. Ductile: Agentic large language model orchestration of engineering analysis in product development design practice. *Journal of Mechanical Design*, 148(12):121706, 07 2026. ISSN 1050-0472. doi:10.1115/1.4072279. URL <https://doi.org/10.1115/1.4072279>.
- [56] Yupeng Qi, Ran Xu, and Xu Chu. FeaGPT: an End-to-End agentic-AI for Finite Element Analysis, October 2025. URL <https://arxiv.org/abs/2510.21993>.
- [57] Yujia Qin, Shihao Liang, Yining Ye, et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=dHng200Jjr>.
- [58] Lyle Regenwetter, Amin Heyrani Nobari, and Faez Ahmed. Deep generative models in engineering design: A review. *Journal of Mechanical Design*, 144(7):071704, 2022. doi:10.1115/1.4053859. URL <https://doi.org/10.1115/1.4053859>.
- [59] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992. Association for Computational Linguistics, 2019. doi:10.18653/v1/D19-1410. URL <https://aclanthology.org/D19-1410/>.
- [60] R.W. Saaty. The analytic hierarchy process—what it is and how it is used. *Mathematical Modelling*, 9(3):161–176, 1987. ISSN 0270-0255. doi:10.1016/0270-0255(87)90473-8. URL <https://www.sciencedirect.com/science/article/pii/0270025587904738>.

- [61] Alexandre Sallinen, Stefan Krsteski, Paul Teiletche, et al. M(M)ORE: Massive multimodal open RAG & extraction. In *Championing Open-source DEvelopment in ML Workshop @ ICML25*, 2025. URL <https://openreview.net/forum?id=6j1HjfIdKn>.
- [62] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 68539–68551. Curran Associates, Inc., 2023. doi:10.52202/075280-2997. URL <https://doi.org/10.52202/075280-2997>.
- [63] O. Sigmund. A 99 line topology optimization code written in matlab. *Struct. Multidiscip. Optim.*, 21(2): 120–127, April 2001. ISSN 1615-147X. doi:10.1007/s001580050176. URL <https://doi.org/10.1007/s001580050176>.
- [64] Patara Trirat, Wonyong Jeong, and Sung Ju Hwang. AutoML-agent: A multi-agent LLM framework for full-pipeline autoML. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=p1UBWk0vZm>.
- [65] Jianguo Wang, Xiaomeng Yi, Rentong Guo, et al. Milvus: A purpose-built vector data management system. In *Proceedings of the 2021 International Conference on Management of Data*, SIGMOD '21, page 2614–2627, New York, NY, USA, 2021. Association for Computing Machinery. ISBN 9781450383431. doi:10.1145/3448016.3457550. URL <https://doi.org/10.1145/3448016.3457550>.
- [66] Zeyu Wang, Frank P.-W. Lo, Qian Chen, et al. An LLM-enabled multi-agent autonomous mechatronics design framework. In *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pages 4205–4215. IEEE, 2025. doi:10.1109/CVPRW67362.2025.00404. URL <https://doi.org/10.1109/CVPRW67362.2025.00404>.
- [67] Zhenting Wang, Qi Chang, Hemani Patel, et al. MCP-Bench: Benchmarking tool-using LLM agents with complex real-world tasks via MCP servers, 2025. URL <https://arxiv.org/abs/2508.20453>. arXiv:2508.20453.
- [68] Hjalmar Wijk, Tao Roa Lin, Joel Becker, Sami Jawhar, Neev Parikh, Thomas Broadley, Lawrence Chan, Michael Chen, Joshua M Clymer, Jai Dhyani, Elena Ericheva, Katharyn Garcia, Brian Goodrich, Nikola Jurkovic, Megan Kinniment, Aron Lajko, Seraphina Nix, Lucas Jun Koba Sato, William Saunders, Maksym Taran, Ben West, and Elizabeth Barnes. RE-bench: Evaluating frontier AI r&d capabilities of language model agents against human experts. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=3rB0bVU6z6>.
- [69] Qingyun Wu, Gagan Bansal, Jieyu Zhang, et al. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. In *First Conference on Language Modeling (COLM)*, 2024. URL <https://openreview.net/forum?id=BAakY1hNKS>. arXiv:2308.08155.
- [70] David C. Wynn and P. John Clarkson. Process models in design and development. *Research in Engineering Design*, 29(2):161–202, 2018. doi:10.1007/s00163-017-0262-7. URL <https://doi.org/10.1007/s00163-017-0262-7>.
- [71] David C. Wynn and Claudia M. Eckert. Perspectives on iteration in design and development. *Research in Engineering Design*, 28(2):153–184, 2017. doi:10.1007/s00163-016-0226-3. URL <https://doi.org/10.1007/s00163-016-0226-3>.
- [72] Zeda Xu, Nikolas Martelaro, and Christopher McComb. Supervising ralph wiggum: Exploring a metacognitive co-regulation agentic AI loop for engineering design, March 2026. URL <https://arxiv.org/abs/2603.24768>.
- [73] An Yang, Anfeng Li, Baosong Yang, et al. Qwen3 Technical Report, May 2025. URL <http://arxiv.org/abs/2505.09388>. arXiv:2505.09388 [cs].
- [74] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.
- [75] Shunyu Yao, Noah Shinn, Pedram Razavi, et al. τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=roNSXZpUDN>.
- [76] Megan Y. Ying, Daniele Grandi, Allin Groom, and Christopher McComb. Aligning agents with experts in material selection: Prompting, size, and reasoning. *Journal of Computing and Information Science in Engineering*, 26(7), May 2026. doi:10.1115/1.4071467. URL <https://doi.org/10.1115/1.4071467>.

[77] Xiyuan Zhou, Xinlei Wang, Yirui He, et al. EngiBench: A Benchmark for Evaluating Large Language Models on Engineering Problem Solving, September 2025. URL <http://arxiv.org/abs/2509.17677>. arXiv:2509.17677 [cs].

A Compute Environment

The Ollama-served models use 8-bit quantization with a 32,768-token context window. The multi-agent system and all local (Ollama) inference run on a MacBook Pro (Apple M4 Pro, 24 GB unified memory); HPC training jobs are submitted to the ETH Zurich Euler cluster on NVIDIA RTX 4090 GPU nodes, whose per-node CPU and memory allocations vary across runs.

The RAG service (MMORE) combines dense and sparse retrieval. Dense retrieval uses 384-dimensional all-MiniLM-L6-v2 embeddings with cosine similarity [59]; sparse retrieval uses SPLADE, a learned sparse lexical-expansion retriever [20], weighted equally ($\alpha = 0.5$). The system retrieves five chunks, reranks them with bge-reranker-base, stores vectors in Milvus [65], and synthesizes answers with Qwen3 8B [73].

B Scoring Methodology

B.1 Design Quality Metrics

The design quality score is a weighted combination of six per-design metrics:

$$S_{\text{design}} = 0.31 \cdot \text{IoU} + 0.19 \cdot \text{PA} + 0.15 \cdot \text{Obj} + 0.12 \cdot \text{Constr} + 0.12 \cdot \text{Conn} + 0.11 \cdot \text{WT} \quad (1)$$

where:

- **IoU** (Intersection over Union): overlap between the binarized agent design and ground-truth design, $\text{IoU} = |A \cap G| / |A \cup G|$, with threshold 0.5.
- **PA** (Pixel Accuracy): fraction of correctly classified pixels, $\text{PA} = \frac{1}{N} \sum_i \mathbf{1}(A_i = G_i)$, where N is the total number of pixels.
- **Obj** (Objective Score): normalized score based on problem-specific objectives (*e.g.*, compliance, field overlap).
- **Constr** (Constraint Score): smooth partial credit for constraint satisfaction, $\exp(-|v_{\text{actual}} - v_{\text{target}}|/\tau)$ with temperature τ , averaged across all constraints.
- **Conn** (2D Connectivity): validates topological connectivity of the 2D design.
- **WT** (3D Watertightness): validates that the exported 3D mesh forms a closed, watertight solid. In practice, WT is uniformly 0.00 across all configurations because the threshold-and-extrude STL export produces non-manifold meshes; a post-processing repair step would be needed for watertight output.

Weights were assigned by engineering judgment, prioritizing geometric fidelity over engineering performance metrics and printability. A post-hoc Analytic Hierarchy Process (AHP) analysis [60] confirmed consistency ($\text{CR} = 0.012$) and produced weights within ± 0.03 of the chosen values.

The hierarchical workflow score combines design quality with operational metrics:

$$S_{\text{workflow}} = 0.65 \cdot S_{\text{design}} + 0.20 \cdot S_{\text{tool}} + 0.15 \cdot S_{\text{completion}} \quad (2)$$

where S_{tool} is the tool-call efficiency, defined as the ratio of correctly matched calls to the maximum of optimal and actual call counts, and $S_{\text{completion}} \in \{0, 1\}$ indicates whether all required tools were called successfully. For styles with STL export (W-RAND through W-MULTI), task completion additionally requires that the export parameters match the prompt specifications within a tolerance of ± 0.05 for floats and exact match for booleans; a successful tool call with incorrect parameters counts as failed. For the NATURAL style, when the agent correctly requests clarification ($S_{\text{completion}} = 1$) and does not produce a design, S_{design} is set to 1.0, reflecting that abstention is the correct behavior for underspecified prompts.

B.2 RAG Scoring Details

The RAG score is a weighted sum over n evaluated parameters plus a retrieval indicator:

$$S_{\text{RAG}} = \sum_{i=1}^n w_i \cdot a_i^{\text{eff}} + w_{\text{RAG}} \cdot \mathbf{1}(\text{RAG called}) \quad (3)$$

where effective accuracy $a_i^{\text{eff}} = a_i$ if the agent called *search_documents* and 0 otherwise, and raw accuracy $a_i = \mathbf{1}(|v_{\text{actual}} - v_{\text{expected}}| \leq \tau)$ with tolerance $\tau = 0.05$ for volume fraction and force distance, $\tau = 0.5$ for filter radius. The per-prompt scoring weights (Table 12) decrease per-parameter as more parameters are evaluated (1 to 3 across prompts), while w_{RAG} shrinks from 0.40 to 0.10 since the gating mechanism becomes increasingly redundant when multiple parameters must all be correct.

Table 12: RAG scoring weights per prompt. Target parameters are given in Table 5.

ID	w_{vol}	w_{force}	w_{rmin}	w_{rag}
P0	0.60	—	—	0.40
P1	0.40	0.40	—	0.20
P2	0.40	—	0.40	0.20
P3	0.30	0.30	0.30	0.10

B.3 HPC Workflow Score

The primary metric is a weighted composite:

$$S_{\text{HPC}} = w_{\text{step}} \cdot \frac{n_{\text{completed}}}{4} + w_{\text{config}} \cdot c_{\text{config}} + w_{\text{eval}} \cdot \min\left(1, \frac{n_{\text{metrics}}}{6}\right) \quad (4)$$

where $n_{\text{completed}}$ is the number of completed workflow steps (out of 4: generate, submit, monitor, evaluate), $c_{\text{config}} \in \{0, 1\}$ indicates whether the training configuration matches the prompt specifications, and n_{metrics} counts the evaluation metrics successfully extracted. The base weights are $w_{\text{step}} = 0.70$, $w_{\text{config}} = w_{\text{eval}} = 0.15$.

Each secondary weight (w_{config} , w_{eval}) is set to zero when its corresponding step was not called, with the unused weight redistributed to w_{step} (i.e., $w_{\text{step}} = 1.0 - w_{\text{config}} - w_{\text{eval}}$). This avoids double penalization: a missing step is already penalized in the step completion rate. The evaluation metrics counted in n_{metrics} are IOG, COG, FOG, MMD, DPP, and RVC.

B.4 Training-Code Score

Figure 11 gives an overview of the benchmark pipeline end to end. The training-code composite for a single trial is the weighted sum

$$S_{\text{tc}} = 0.55q + 0.15c + 0.10j + 0.10d + 0.05s + 0.05l, \quad (5)$$

where q is model quality, c code quality, j job completion, d the one-epoch local dry run, s syntax validity, and l SLURM acceptance. All terms lie in $[0, 1]$, and every term except q is binary. Code quality $c = 1$ when the static validator’s structural contracts—the cross-file symbol, signature, checkpoint-key, and artifact-name checks—together with its forward-pass check all pass; s marks a syntactically valid script; l marks a submission script accepted by `sbatch --test-only`; and j and d mark, respectively, a completed full training job on the cluster and a successful one-epoch local execution of the authored script. The weights prioritize the graded design-quality term q over the binary operational gates; a post-hoc AHP analysis [60] confirmed their consistency ($\text{CR} = 0.004$) and produced weights within ± 0.03 of the chosen values.

The model-quality term q compares the agent-trained model with the reference EngiOpt model trained using the same seed. If the trial produces a scorable model and a matched reference is available, then

$$q = \min\left(1, \frac{\text{MMD}_{\text{ref}}}{\text{MMD}_{\text{agent}}}\right).$$

The authored architecture is therefore not required to match the reference implementation. This procedure permits evaluation of the agent-selected architecture at T6v2.

Before model quality is scored, the sampling function is tested for degeneracy. A sampler receives $q = 0$ only when its outputs are both nearly constant across latent samples and insensitive to changes in conditioning. Requiring both conditions distinguishes this failure from low variation along only one of these dimensions.

B.5 Generative Model Quality Metrics

For the HPC training and training-code benchmarks, we report the following EngiOpt metrics [19] to evaluate trained generative models against the dataset baseline. Let $\mathcal{D}$ denote the dataset designs and $\mathcal{D}_g$ the generated designs.

- **MMD** —Distributional similarity between generated and dataset designs (lower is better):

$$\text{MMD}^2(k, \mathcal{D}, \mathcal{D}_g) = \mathbb{E}[k(z, z')] + \mathbb{E}[k(\hat{z}, \hat{z}')] - 2\mathbb{E}[k(z, \hat{z})] \quad (6)$$

where $z, z' \sim \mathcal{D}$, $\hat{z}, \hat{z}' \sim \mathcal{D}_g$, and $k(a, b) = \exp(-\|a - b\|^2 / (2\sigma^2))$ is a Gaussian kernel with $\sigma = 10.0$.

- **DPP** —Diversity of generated designs (higher is better):

$$\text{DPP}(\mathcal{D}_g) = \det(K) \quad (7)$$

where $K \in \mathbb{R}^{M \times M}$ with $K_{ij} = k(\hat{x}_i, \hat{x}_j)$ for $\hat{x}_i, \hat{x}_j \in \mathcal{D}_g$. In practice, K is regularized as $K + 10^{-6}I$ for numerical stability. In every HPC and training-code evaluation, each trained model generates $M=50$ designs per seed for the MMD/DPP computation.

- **COG** —Sum of optimality gaps across the optimization path (lower is better):

$$\text{COG}(x_0) = \sum_{t=0}^T [\tilde{f}(x_t, c) - f^*] \quad (8)$$

where $f^* = \max_{x \in \mathcal{X}} \tilde{f}(x, c)$ is the optimal objective value over the feasible design space $\mathcal{X}$, c denotes the conditioning context (e.g., boundary conditions and loads), x_0 is the generated design refined through T optimization steps, and $\tilde{f}$ denotes the simulated objective.

- **RVC** —Fraction of designs violating at least one constraint (lower is better):

$$\text{RVC}(\mathcal{D}_g) = \frac{1}{M} \sum_{k=1}^M \mathbf{1}(\exists i : g_i(\hat{x}_k, a(\hat{x}_k), c) > 0 \text{ or } \exists j : h_j(\hat{x}_k, a(\hat{x}_k), c) \neq 0) \quad (9)$$

where $M = |\mathcal{D}_g|$ is the number of generated designs, $a(\hat{x}_k)$ denotes the attributes of design $\hat{x}_k$, c the conditions, g_i and h_j are inequality and equality constraints, respectively, and $\mathbf{1}(\cdot)$ is the indicator function.

- **IOG** —The $t=0$ term of the COG sum, i.e., $\tilde{f}(x_0, c) - f^*$.
- **FOG** —The $t=T$ term of the COG sum, i.e., $\tilde{f}(x_T, c) - f^*$.

C Benchmark Prompts

Representative prompts for each evaluation dimension. Workflow prompts use Beams2D with volfrac=0.4, forcedist=0.65, rmin=4.0, seed=42; the seven prompt styles target distinct cognitive demands as described in Section 3.4.1. HPC training and RAG evaluation prompts follow.

C.1 Workflow Prompts

FULL

Design a 2D beam structure.

Design requirements:

- Use a material volume fraction of 0.4
- Force distance parameter: 0.65
- Minimum filter radius (rmin): 4.0

Optimize the structure and simulate the result to obtain the compliance value.

NATURAL

Design a 2D beam structure.

Design requirements:

- The design should be moderate material usage
 - Apply a force distributed in the upper right region
- Optimize the structure and simulate the result to obtain the compliance value.
-

W-RAND

Execute a 2D topology optimization, simulate the result, and export the geometry as a 3D-printable STL file.

1. Optimization Configuration
 - Volume Fraction: 0.4
 - Force Distance: 0.65
 - Filter Radius (rmin): 4.0
 - Objective: Minimize compliance
 2. Simulation
 - After optimization, simulate the design to obtain the compliance value
 3. Post-processing & Export
 - Thresholding: Apply a 0.58 density threshold to convert the continuous density map into binary geometry
 - Mirror: Mirror the design across the y-axis for the final geometry
 - XY Scaling: Scale the X and Y dimensions by 2.47
 - Extrusion: Extrude the 2D result by 17.9 units in the Z-axis to create a 3D volume
 - Export: Save the final geometry as an STL file with these exact parameters
-

W-DERIVED

Execute a 2D topology optimization, simulate the result, and export the geometry as a 3D-printable STL file.

1. Optimization Configuration
 - Volume Fraction: 0.4
 - Force Distance: 0.65
 - Filter Radius (rmin): 4.0
 - Objective: Minimize compliance
 2. Simulation
 - After optimization, simulate the design to obtain the compliance value
 3. Post-processing & Export

The STL export parameters must be derived from the optimization inputs:

 - Thresholding: Use the volume fraction value as the density threshold
 - Mirror: Mirror the design across the y-axis only if the volume fraction is greater than 0.4
 - XY Scaling: Scale the X and Y dimensions by twice the filter radius
 - Extrusion: Extrude the 2D result in the Z-axis by the threshold value multiplied by 40
 - Export: Save the final geometry as an STL file with these derived parameters
-

W-DISTRACT

Execute a 2D topology optimization, simulate the result, and export the geometry as a 3D-printable STL file.

1. Optimization Configuration
 - Volume Fraction: 0.4
 - Force Distance: 0.65
 - Filter Radius (rmin): 4.0
 - Objective: Minimize compliance
 2. Simulation
 - After optimization, simulate the design to obtain the compliance value
 3. Post-processing & Export
 - Threshold the density field at 0.42 to preview the design topology
 - Apply a 0.58 density threshold to produce the final solid/void geometry
 - Scale the preview display by 1.76x in XY for quick inspection
 - Scale the X and Y dimensions of the part by 2.47 for manufacturing
 - Mirror the design across the y-axis for the final geometry
 - Extrude the 2D result by 17.9 units in the Z-axis to create a 3D volume
 - Export: Save the final geometry as an STL file with these exact parameters
-

W-COND

Execute a 2D topology optimization, simulate the result, then export the geometry as a 3D-printable STL file with parameters that depend on the simulation outcome.

1. Optimization Configuration
 - Volume Fraction: 0.4
 - Force Distance: 0.65
 - Filter Radius (rmin): 4.0
 - Objective: Minimize compliance
 2. Simulation
 - After optimization, simulate the design to obtain the compliance value
 3. Post-processing & Export (conditional on compliance)
 - If compliance > 254.8:
 - Thresholding: Apply a 0.48 density threshold to convert the continuous density map into binary geometry
 - Mirror: Mirror the design across the y-axis for the final geometry
 - If compliance <= 254.8:
 - Thresholding: Apply a 0.64 density threshold to convert the continuous density map into binary geometry
 - Mirror: Do NOT mirror the design for the final geometry
 - In both cases:
 - XY Scaling: Scale the X and Y dimensions by 0.92
 - Extrusion: Extrude the 2D result by 19.6 units in the Z-axis to create a 3D volume
 - Export: Save the final geometry as an STL file with these exact parameters
-

W-MULTI

Execute a 2D topology optimization, simulate the result, and export the geometry as TWO separate 3D-printable STL files with different parameters.

1. Optimization Configuration
 - Volume Fraction: 0.4
 - Force Distance: 0.65
 - Filter Radius (rmin): 4.0
 - Objective: Minimize compliance
 2. Simulation
 - After optimization, simulate the design to obtain the compliance value
 3. Post-processing & Export
 - Export A:
 - Thresholding: Apply a 0.48 density threshold to convert the continuous density map into binary geometry
 - Mirror: Mirror the design across the y-axis for the final geometry
 - XY Scaling: Scale the X and Y dimensions by 3.64
 - Extrusion: Extrude the 2D result by 19.6 units in the Z-axis to create a 3D volume
 - Export: Save the final geometry as an STL file with these exact parameters
 - Export B:
 - Thresholding: Apply a 0.64 density threshold to convert the continuous density map into binary geometry
 - Mirror: Do NOT mirror the design for the final geometry
 - XY Scaling: Scale the X and Y dimensions by 0.92
 - Extrusion: Extrude the 2D result by 16.4 units in the Z-axis to create a 3D volume
 - Export: Save the final geometry as an STL file with these exact parameters
-

C.2 RAG Evaluation Prompts

P0

The EngiBench paper documents the default design conditions for the Beams2D problem in its API walkthrough.

Search the paper to find the default volume fraction (volfrac) listed for the Beams2D problem. Then generate a 2D beam design using exactly that volume fraction. Use default values for all other parameters (do NOT ask for clarification -- proceed directly with defaults).

P1

In the EngiBench paper's Section 3.1 API walkthrough, a code example runs a Beams2D optimization using non-default design conditions. Search the paper to find both the volume fraction and force distance from that example, then generate a 2D beam design with those exact values. Use default values for all other parameters and do not ask for clarification.

P2

The SOPTX paper by He et al. (2025) benchmarks its topology optimization framework on a 2D cantilever beam problem.

Search the paper to find both the volume fraction (volfrac) and the filter radius (rmin) used for that 2D cantilever benchmark. Then generate a 2D beam design using exactly those values. Use default values for all other parameters and do not ask for clarification.

P3

Generate a 2D beam design combining parameters from multiple sources:

1. Use the volume fraction and force distance from the EngiBench paper’s API walkthrough example (the non-default values shown in the code snippet).
2. Use the filter radius from the SOPTX paper by He et al. (2025) for their 2D cantilever beam benchmark.

Search the relevant papers to find each value, then generate a 2D beam design using exactly those three parameters. Use default values for all other parameters and do not ask for clarification.

C.3 HPC Training Prompts

HPC-TRAIN (EXPLICIT)

Train a cGAN CNN 2D generative model for the Beams2D topology optimization problem on the Euler HPC cluster, then evaluate it against the dataset baseline using the standard EngiOpt evaluation script.

Step 1: Generate Training Script

- Use the generate_training_command tool with:
 algorithm: cgan_cnn_2d
 problem_id: beams2d
 epochs: 100
 seed: 1

Step 2: Submit to HPC

- Submit the generated SLURM script to the Euler cluster

Step 3: Monitor Training

- Monitor the job until it completes
- Use check_interval=30 and max_checks=200 for the monitoring

Step 4: Evaluate Trained Model

- Use the evaluate_model tool to evaluate the trained model against the dataset baseline:
 problem_id: beams2d
 algorithm: cgan_cnn_2d
 seed: 1
 n_samples: 50
- This downloads the model from WandB, generates designs, and computes metrics (IOG, COG, FOG, MMD, DPP, violation rate)
- Report the evaluation metrics from the output

Complete all steps in order. Do not ask for clarification.

HPC-TRAIN (NATURAL)

Train a cGAN CNN 2D model for the Beams2D topology optimization problem on the Euler HPC cluster with seed 1 and 100 epochs. Use the available tools to generate the SLURM training script -- do not write or modify any scripts manually. Submit the job and wait for it to finish. Then use the model evaluation tool to evaluate the trained model against the dataset -- it will download the model from WandB automatically. Report the metrics.

Do not ask for clarification.

C.4 Training-Code Authoring Prompts

Representative prompts (seed 1) for six training-code authoring tiers and variants (T2, T3, T4, T4-Func, T5, T6v2). The shared blanked EngiOpt cGAN training file embedded in each prompt is elided; Figure 2 shows which regions each tier authors.

TC-T2

You are given a partially-blanked EngiOpt training file for the Beams2D topology-optimization problem. Complete the blanked regions so the file trains a working conditional generative model, then train and evaluate it on the Euler HPC cluster.

Tier: T2 (plumbing). Blanked regions: B6, B7.

Contract:

Fill the blanked plumbing (sampling helper, logging) so the training loop runs end-to-end on Beams2D.

Training file to complete (fill every BLANK; keep all preserved code verbatim, including the 'Generator = NetG' alias and the checkpoint block):

[380-line embedded training-file scaffold omitted -- see the tier-explainer figure]

After completing the file, validate it locally (validation runs a 1-epoch local dry-run), then submit a single full training job to the Euler HPC cluster with seed 1 -- do not submit a separate cluster dry-run job. Monitor the job to completion. Then evaluate the trained model by calling the 'evaluate_model' tool (which runs the EngiBench metric harness) -- do NOT compute your own metrics. Report the metrics it returns. Do not ask for clarification.

TC-T3

You are given a partially-blanked EngiOpt training file for the Beams2D topology-optimization problem. Complete the blanked regions so the file trains a working conditional generative model, then train and evaluate it on the Euler HPC cluster.

Tier: T3 (training loop). Blanked regions: B4a, B4b, B5.

Contract:

Fill the blanked training loop (BCE loss, two Adam optimizers, the alternating cGAN generator/discriminator update).

Training file to complete (fill every BLANK; keep all preserved code verbatim, including the 'Generator = NetG' alias and the checkpoint block):

[401-line embedded training-file scaffold omitted -- see the tier-explainer figure]

After completing the file, validate it locally (validation runs a 1-epoch local dry-run), then submit a single full training job to the Euler HPC cluster with seed 1 -- do not submit a separate cluster dry-run job. Monitor the job to completion. Then evaluate the trained model by calling the 'evaluate_model' tool (which runs the EngiBench metric harness) -- do NOT compute your own metrics. Report the metrics it returns. Do not ask for clarification.

TC-T4

You are given a partially-blanked EngiOpt training file for the Beams2D topology-optimization problem. Complete the blanked regions so the file trains a working conditional generative model, then train and evaluate it on the Euler HPC cluster.

Tier: T4 (architecture). Blanked regions: B3.G.init, B3.G.forward, B3.D.init, B3.D.forward.

Contract:

Implement the blanked NetG and NetD CNN class bodies; honour the constructor signatures and forward-pass shape contract that remain.

Training file to complete (fill every BLANK; keep all preserved code verbatim, including the 'Generator = NetG' alias and the checkpoint block):

[348-line embedded training-file scaffold omitted -- see the tier-explainer figure]

After completing the file, validate it locally (validation runs a 1-epoch local dry-run), then submit a single full training job to the Euler HPC cluster with seed 1 -- do not submit a separate cluster dry-run job. Monitor the job to completion. Then evaluate the trained model by calling the 'evaluate_model' tool (which runs the EngiBench metric harness) -- do NOT compute your own metrics. Report the metrics it returns. Do not ask for clarification.

TC-T4-FUNC

You are given a partially-blanked EngiOpt training file for the Beams2D topology-optimization problem. Complete the blanked regions so the file trains a working conditional generative model, then train and evaluate it on the Euler HPC cluster.

Tier: T4-FUNC (architecture + sampler). Blanked regions: B3.G.init, B3.G.forward, B3.D.init, B3.D.forward, B9.sample.

Contract:

Implement the blanked NetG and NetD CNN class bodies, AND author the blanked top-level sample(model, conditions_tensor, n_samples, latent_dim, design_shape, device) that returns n_samples designs of shape (n_samples, *design_shape) in [0, 1]. Your design need not reproduce the canonical architecture exactly -- it is scored on the quality of the designs your own sampler produces.

Training file to complete (fill every BLANK; keep all preserved code verbatim, including the 'Generator = NetG' alias and the checkpoint block):

[352-line embedded training-file scaffold omitted -- see the tier-explainer figure]

After completing the file, validate it locally with ‘validate_training_script(require_sample=True)’ (validation runs a 1-epoch local dry-run AND checks your sample() adapter), then submit a single full training job to the Euler HPC cluster with seed 1 -- do not submit a separate cluster dry-run job. Monitor the job to completion. Then evaluate by calling ‘evaluate_model(eval_mode=’functional’, candidate_path=<the file you saved with write_training_script>)’ -- it loads YOUR Generator class and runs YOUR sample() adapter, then scores the resulting designs with the EngiBench metric oracle -- do NOT compute your own metrics. Report the metrics it returns. Do not ask for clarification.

TC-T5

You are given a partially-blanked EngiOpt training file for the Beams2D topology-optimization problem. Complete the blanked regions so the file trains a working conditional generative model, then train and evaluate it on the Euler HPC cluster.

Tier: T5 (whole module (stretch)). Blanked regions: B1c, B3.G.init, B3.G.forward, B3.D.init, B3.D.forward, B4a, B4b, B5, B6, B7.

Contract:

Complete the whole module from the skeleton into a working conditional CNN-GAN.

Training file to complete (fill every BLANK; keep all preserved code verbatim, including the ‘Generator = NetG’ alias and the checkpoint block):

[272-line embedded training-file scaffold omitted -- see the tier-explainer figure]

After completing the file, validate it locally (validation runs a 1-epoch local dry-run), then submit a single full training job to the Euler HPC cluster with seed 1 -- do not submit a separate cluster dry-run job. Monitor the job to completion. Then evaluate the trained model by calling the ‘evaluate_model’ tool (which runs the EngiBench metric harness) -- do NOT compute your own metrics. Report the metrics it returns. Do not ask for clarification.

The uppercase term NOVEL in the following prompt is retained verbatim from the prompt used in the evaluation. In this context, it instructs the agent to select a model architecture that is not constrained to the supplied reference cGAN. It does not denote or require an algorithm that is novel relative to the research literature.

TC-T6v2

Design and implement a NOVEL generative training algorithm for the Beams2D topology-optimization problem, from scratch -- there is NO template to fill in.

Tier: T6V2 (novel algorithm + self-declared interface).

Interface specification (your training module MUST satisfy this):

Design a NOVEL conditional generative model for Beams2D from scratch -- ANY family (GAN / VAE / normalizing flow / diffusion / energy-based / your own). Unlike the fixed-cGAN tier, you DECLARE your OWN interface via a top-level ‘CONTRACT = {...)’

literal dict the eval reads; your module MUST satisfy what it declares:

- CONTRACT (a literal dict, values must be plain literals) with fields: family, model_class, conditioning_layout, checkpoint_filename, checkpoint_key, artifact_suffix, sample_args, latent_dim.
- a top-level class named by CONTRACT[‘model_class’] (an nn.Module) with ‘__init__(latent_dim, n_conds, design_shape)’ -- this EXACT positional signature, because the eval builds it as model_class(latent_dim, n_conds, design_shape).
- a top-level ‘sample(...)’ returning (n_samples, *design_shape) in [0, 1]; its parameter names (declared, in order, in CONTRACT[‘sample_args’]) are drawn from {conditions, conditions_tensor, design_shape, device, latent_dim, model, n_samples} -- the eval binds them BY NAME (a subset / reordering is fine; ‘conditions’ and ‘conditions_tensor’ are the SAME routed tensor, so pick ONE).
- CONTRACT[‘conditioning_layout’] is one of {flat_2d, none, seq_3d, spatial_4d}: the eval reshapes the raw (B, n_conds) conditions into it before your sample() (spatial_4d=(B,n,1,1), flat_2d=(B,n), seq_3d=(B,1,n), none=unchanged).
- checkpoint (when --save-model): ‘th.save({“<checkpoint_key>”: model.state_dict(), “<checkpoint_filename>”})’, then log a wandb artifact ‘f”{args.problem_id}_{args.algo}<artifact_suffix>”’ with alias ‘f”seed_{args.seed}”’. Use the STRING LITERALS matching your CONTRACT (checkpoint_key / checkpoint_filename / artifact_suffix) -- the validator scans the source statically. Everything sample() needs must live in that ONE checkpointed nn.Module.
- log ‘latent_dim’ to the wandb run config (the eval reads it back from run.config, not the checkpoint).

Train it on Beams2D. You are scored on the QUALITY of the designs your sampler produces (via evaluate_model), NOT on matching any reference.

Provided I/O harness -- use this CLI / dataset-loading / checkpoint plumbing VERBATIM (it is the boilerplate every engiopt script shares; do NOT reinvent or reverse-engineer the EngiBench dataset API). Design your OWN Generator, sample(), loss, and training loop in the marked section -- that is where the novelty must be:

[194-line embedded training-file scaffold omitted -- see the tier-explainer figure]

Write the complete training script with ‘write_training_script’, then validate it locally with ‘validate_training_script(require_sample=True)’ (validation runs a 1-epoch local dry-run AND checks your sample() adapter), then submit a single full training job to the Euler HPC cluster with seed 1 -- do not submit a separate cluster dry-run job. Monitor the job to completion. Then evaluate by calling ‘evaluate_model(eval_mode=’functional’, candidate_path=<the file you saved with write_training_script>)’ -- it loads YOUR Generator class and runs YOUR sample() adapter, then scores the resulting designs with the EngiBench metric oracle -- do NOT compute your own metrics. Report the metrics it returns. Do not ask for clarification.

D Additional Results

D.1 Statistical Analysis

We summarize the combined-overall scores using medians and 95% percentile-bootstrap confidence intervals. Each interval is calculated from 9,999 resamples of the trials within that cell. We use non-parametric procedures because the scores are bounded in $[0, 1]$ and contain many tied values. Table 13 reports the resulting medians and confidence intervals.

All comparisons preserve the pairing of trials by dataset sample and random seed. Within each prompt style, model pairs are compared using the Wilcoxon signed-rank test. Within each model backend, adjacent prompt styles are compared using the same test. We report the rank-biserial correlation to indicate the direction and magnitude of each paired difference. The Benjamini–Hochberg procedure controls for multiple comparisons separately within the model-comparison and style-comparison families. We additionally use the Friedman test [21] as an uncorrected screening test for differences among prompt styles within each backend. Where a prompt ordering was specified in advance, the Page test evaluates whether the scores follow that ordering.

Table 13: Per-cell median combined-overall score with 95% bootstrap confidence intervals for the workflow benchmark (percentile bootstrap over the n trials in each cell; n below 15 reflects the lost Qwen3.5-4B runs noted in Section 4).

Model	Style	n	Median	95% CI
<i>Photonics2D</i>				
GPT-5-mini	W-RAND	15	0.528	[0.520, 0.541]
GPT-5-mini	W-DISTRACT	15	0.522	[0.511, 0.535]
GPT-5-mini	W-COND	15	0.452	[0.394, 0.528]
Gemini-3-Flash	W-RAND	15	0.597	[0.592, 0.612]
Gemini-3-Flash	W-DISTRACT	15	0.547	[0.537, 0.551]
Gemini-3-Flash	W-COND	15	0.593	[0.447, 0.613]
Qwen3-4B	W-RAND	15	0.550	[0.542, 0.565]
Qwen3-4B	W-DISTRACT	15	0.514	[0.511, 0.531]
Qwen3-4B	W-COND	15	0.397	[0.385, 0.411]
Qwen3.5-4B	W-RAND	15	0.547	[0.541, 0.557]
Qwen3.5-4B	W-DISTRACT	15	0.552	[0.540, 0.608]
Qwen3.5-4B	W-COND	15	0.465	[0.402, 0.559]
<i>Beams2D</i>				
GPT-5-mini	W-RAND	15	0.674	[0.599, 0.717]
GPT-5-mini	W-DERIVED	15	0.646	[0.594, 0.694]
GPT-5-mini	W-DISTRACT	15	0.637	[0.599, 0.668]
GPT-5-mini	W-COND	15	0.694	[0.606, 0.739]
GPT-5-mini	W-MULTI	15	0.696	[0.595, 0.744]
Gemini-3-Flash	W-RAND	15	0.646	[0.586, 0.689]
Gemini-3-Flash	W-DERIVED	15	0.612	[0.586, 0.694]
Gemini-3-Flash	W-DISTRACT	15	0.629	[0.556, 0.689]
Gemini-3-Flash	W-COND	15	0.662	[0.622, 0.739]
Gemini-3-Flash	W-MULTI	15	0.696	[0.674, 0.759]
Qwen3-4B	W-RAND	15	0.637	[0.599, 0.700]
Qwen3-4B	W-DERIVED	15	0.518	[0.487, 0.582]
Qwen3-4B	W-DISTRACT	15	0.599	[0.557, 0.664]
Qwen3-4B	W-COND	15	0.556	[0.475, 0.700]
Qwen3-4B	W-MULTI	15	0.647	[0.566, 0.740]
Qwen3.5-4B	W-RAND	14	0.684	[0.662, 0.733]
Qwen3.5-4B	W-DERIVED	13	0.681	[0.606, 0.697]
Qwen3.5-4B	W-DISTRACT	15	0.637	[0.624, 0.748]
Qwen3.5-4B	W-COND	15	0.662	[0.486, 0.739]
Qwen3.5-4B	W-MULTI	14	0.683	[0.662, 0.742]

On Photonics2D, 14 of the 18 pairwise model comparisons remain significant after BH correction ($q < 0.05$). Gemini-3-Flash has higher scores than the other three backends on the random and conditional styles and higher scores than GPT-5-mini and Qwen3-4B on the distractor style (rank-biserial correlation ≥ 0.75 , $q < 0.02$). Qwen3.5-4B has higher scores than Qwen3-4B on the distractor and conditional styles ($q < 0.02$) and higher scores than GPT-5-mini on the distractor and random styles ($q < 0.004$).

The conditional style produces lower scores than the distractor style for GPT-5-mini and both Qwen models ($q < 0.04$). The Friedman screening test also identifies an effect of prompt style for three of the four backends ($p < 0.05$).

On Beams2D, none of the 30 pairwise model comparisons remains significant after correction. Most median scores are near 0.6–0.7, and the observations contain many ties, which limits the power of the paired tests. The absence of significant pairwise differences should therefore not be interpreted as evidence that the models are equivalent. The Friedman test identifies an effect of prompt style for three backends, but the Page test does not support the predefined monotonic ordering of prompt difficulty ($p > 0.7$).

Table 14 reports the retrieval results. With a populated index (RAG-on), the median score is 1.00 for GPT-5-mini and Gemini-3-Flash and 0.40 for Qwen3-4B. With an empty index (Empty RAG), the median score is 0.20 for all three backends. Because both conditions provide access to the retrieval tool, their comparison isolates the effect of making the indexed documents available.

RAG-off has a median score of zero by construction because the scorer requires a retrieval call before awarding parameter credit. This condition verifies the scoring gate but does not provide statistical evidence that indexed retrieval improves parameter selection.

Each model-condition cell was designed to contain 12 trials: four prompts with three repetitions each. One GPT-5-mini RAG-off trial is missing, leaving $n = 11$ in that cell. The pooled summaries combine prompts with different parameter sources and retrieval requirements. They should therefore be interpreted descriptively rather than as estimates for a homogeneous task population.

Table 14: Median combined-overall score with 95% bootstrap confidence intervals for the RAG benchmark. The planned design contains twelve trials per cell (four prompts with three repetitions); GPT-5-mini RAG-off contains eleven. RAG-off is identically zero because the gated scorer denies parameter credit without a retrieval call.

Model	Retrieval	n	Median	95% CI
<i>GPT-5-mini</i>				
GPT-5-mini	RAG-on	12	1.000	[0.850, 1.000]
GPT-5-mini	Empty-RAG	12	0.200	[0.150, 0.300]
GPT-5-mini	RAG-off	11	0.000	[0.000, 0.000]
<i>Gemini-3-Flash</i>				
Gemini-3-Flash	RAG-on	12	1.000	[0.600, 1.000]
Gemini-3-Flash	Empty-RAG	12	0.200	[0.150, 0.600]
Gemini-3-Flash	RAG-off	12	0.000	[0.000, 0.000]
<i>Qwen3-4B</i>				
Qwen3-4B	RAG-on	12	0.400	[0.300, 0.700]
Qwen3-4B	Empty-RAG	12	0.200	[0.150, 0.300]
Qwen3-4B	RAG-off	12	0.000	[0.000, 0.000]

Table 15 reports the HPC results over ten trials per cell. Gemini-3-Flash has a median S_{HPC} of 1.00 for all three evaluated prompt conditions. For GPT-5-mini, the median decreases from 1.00 with the explicit cGAN prompt to 0.89 with the natural cGAN prompt. Its median is 0.81 for the natural diffusion prompt; diffusion was not evaluated with an explicit prompt. The wider GPT-5-mini intervals reflect greater variation among trials.

Table 15: Per-cell median HPC workflow score S_{HPC} with 95% bootstrap confidence intervals (percentile bootstrap over the ten samples per cell).

Model	Prompt	n	Median	95% CI
<i>GPT-5-mini</i>				
GPT-5-mini	Explicit, cGAN	10	1.000	[0.787, 1.000]
GPT-5-mini	Natural, cGAN	10	0.894	[0.575, 1.000]
GPT-5-mini	Natural, diffusion	10	0.806	[0.787, 1.000]
<i>Gemini-3-Flash</i>				
Gemini-3-Flash	Explicit, cGAN	10	1.000	[1.000, 1.000]
Gemini-3-Flash	Natural, cGAN	10	1.000	[1.000, 1.000]
Gemini-3-Flash	Natural, diffusion	10	1.000	[1.000, 1.000]

Table 16 reports the training-code results over ten seeds per cell. Several T4 and T5 model-quality distributions are bimodal: successfully evaluated checkpoints receive scores near the upper end of the range, whereas checkpoints that cannot be loaded receive zero model-quality credit. The medians and bootstrap intervals provide only a partial summary of these distributions; Figure 8 and Figure 14 show the individual seed-level results.

For GPT-5-mini, the composite score differs across T2–T5 (Friedman $p = 0.046$), and the Page test supports a decrease over these tiers ($p = 0.016$). Gemini-3-Flash also decreases from T2 through T5, but its score increases again at T6v2. Although the Page test over T2–T5–T6v2 gives $p = 0.031$, the observed sequence is not monotonic. We therefore do not interpret this test as evidence of a general decline with increasing authoring scope.

The code-quality component is constant across the Gemini-3-Flash tiers (pooled standard deviation 0.00), whereas the model-quality component varies across tiers for both backends (pooled standard deviation 0.36–0.45). For GPT-5-mini, the model-quality component differs significantly among tiers (Friedman $p = 0.033$). Thus, most of the variation in the composite score arises from model quality rather than the operational components.

The weight-sensitivity analysis recomputes and ranks the fourteen model–tier cells under alternative composite weights. The rankings remain similar when model quality is increased or decreased (Kendall $\tau \geq 0.87$) and when the operational components receive greater weight ($\tau = 1.0$). Removing model quality changes the ranking substantially ($\tau = 0.24$). The conclusions are therefore stable under the tested weight changes, except when model quality is omitted entirely.

Table 16: Per-cell median composite score S_{ic} with 95% bootstrap confidence intervals for the training-code benchmark (percentile bootstrap over the ten seeds). Beams2D spans all seven tier variants; Photonics2D covers the plumbing (T2), architecture (T4), and open-synthesis (T6v2) tiers.

Model	Tier	n	Median	95% CI
<i>Beams2D</i>				
GPT-5-mini	T2	10	0.975	[0.732, 1.000]
GPT-5-mini	T3	10	0.976	[0.560, 1.000]
GPT-5-mini	T4	10	0.450	[0.450, 0.725]
GPT-5-mini	T4-func	10	0.950	[0.557, 1.000]
GPT-5-mini	T5	10	0.450	[0.450, 1.000]
GPT-5-mini	T5-func	10	0.400	[0.400, 0.425]
GPT-5-mini	T6v2	10	0.867	[0.619, 0.950]
Gemini-3-Flash	T2	10	1.000	[0.776, 1.000]
Gemini-3-Flash	T3	10	0.924	[0.724, 1.000]
Gemini-3-Flash	T4	10	0.734	[0.590, 1.000]
Gemini-3-Flash	T4-func	10	0.765	[0.564, 1.000]
Gemini-3-Flash	T5	10	0.641	[0.535, 1.000]
Gemini-3-Flash	T5-func	10	0.688	[0.516, 1.000]
Gemini-3-Flash	T6v2	10	0.950	[0.860, 0.950]
<i>Photonics2D</i>				
GPT-5-mini	T2	10	0.969	[0.934, 0.996]
GPT-5-mini	T4	10	0.450	[0.400, 0.950]
GPT-5-mini	T6v2	10	0.950	[0.950, 0.950]
Gemini-3-Flash	T2	10	0.996	[0.950, 1.000]
Gemini-3-Flash	T4	10	1.000	[0.450, 1.000]
Gemini-3-Flash	T6v2	10	0.950	[0.950, 0.950]

D.2 Diffusion Model Results

Figure 12 shows the offline model quality metrics for agent-trained diffusion models, analogous to the cGAN results in Figure 7. Agent-trained diffusion models achieve comparable values to the EngiBench baselines.

D.3 Photonics2D

D.3.1 Workflow Benchmark

Table 17 and Figure 13 provide additional Photonics2D W-COND results for those discussed in Section 4.

D.3.2 Training-code Benchmark

Figure 14 shows the per-seed graded model quality by tier for the training-code benchmark on Photonics2D (Section 4.4).

Table 17: Tool-calling counts for the Photonics2D W-COND prompt style: average calls per tool per sample (mean $\pm$ std). Cell shading encodes call frequency using a linear scale from the smallest to the largest average in the table.

Tool	W-COND			
	GPT-5-mini	Gemini-3-Flash	Qwen3-4B-Q8	Qwen3.5-4B-Q8
optimize_design	1.0 $\pm$ 0.0	1.0 $\pm$ 0.0	1.0 $\pm$ 0.0	1.0 $\pm$ 0.0
simulate_design	1.0 $\pm$ 0.0	1.0 $\pm$ 0.0	1.0 $\pm$ 0.0	1.0 $\pm$ 0.0
convert_design_to_stl	1.1 $\pm$ 0.3	1.0 $\pm$ 0.0	0.5 $\pm$ 0.5	1.0 $\pm$ 0.0
create_problem	0.6 $\pm$ 0.5	0.0 $\pm$ 0.0	1.0 $\pm$ 0.0	0.7 $\pm$ 0.4
render_design	0.3 $\pm$ 0.4	0.0 $\pm$ 0.0	0.1 $\pm$ 0.2	0.0 $\pm$ 0.0

Figure 15 shows designs sampled from the open-synthesis (T6v2) generators authored by the two backends for Photonics2D. This figure is the cross-problem counterpart of the Beams2D results in Figure 10.

For the illustrated seed, both agent-authored generators produce near-binary patterns that vary across the six conditioning vectors. The reference cGAN produces visually similar outputs across these conditions. This example is illustrative; Section 4.4 reports the corresponding quantitative results across ten seeds.

D.3.3 Cross-Problem Portability

We evaluate cross-problem portability using the trained T6v2 scripts. For each trial, we recover the Weave-logged authoring call whose model passes strict loading of that trial’s checkpoint. This produces 20 scripts per source problem: ten from each model backend. T2 and T4 are excluded because they complete complete regions of a shared problem-independent program skeleton.

When executed unchanged on the other problem, 12 of the 20 Beams2D scripts run successfully on Photonics2D, and 14 of the 20 Photonics2D scripts run successfully on Beams2D. Every failure results from a dimension fixed for the source problem: a flattened feature dimension of 9,216 where Photonics2D requires 28,800 in one direction, or a feature-map width of 48 where Beams2D requires 50 in the other.

Portability differs by model backend. All 20 GPT-5-mini scripts transfer successfully because their fully connected architectures derive their dimensions from the input design shape. All transfer failures occur in Gemini-3-Flash scripts whose convolutional architectures contain spatial dimensions fixed for the source problem.

E Raw Benchmark Scores

Tables 18 and 19 report all individual sub-scores per model and prompt style, enabling readers to recompute composite scores under alternative weighting schemes.

Tables 20 and 21 do the same for the training-code benchmark, reporting the six weighted components of S_{tc} separately alongside the composite. Any alternative weighting of those components can therefore be evaluated without re-running the grid.

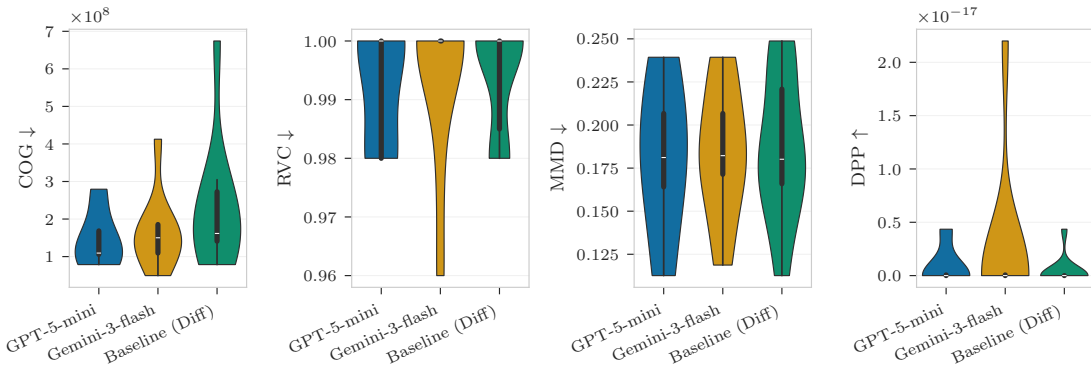


Figure 12: Offline model quality metrics (COG, RVC, MMD, DPP) for agent-trained diffusion models vs. EngiBench baselines. Arrows indicate desired direction. Values averaged across available seeds.

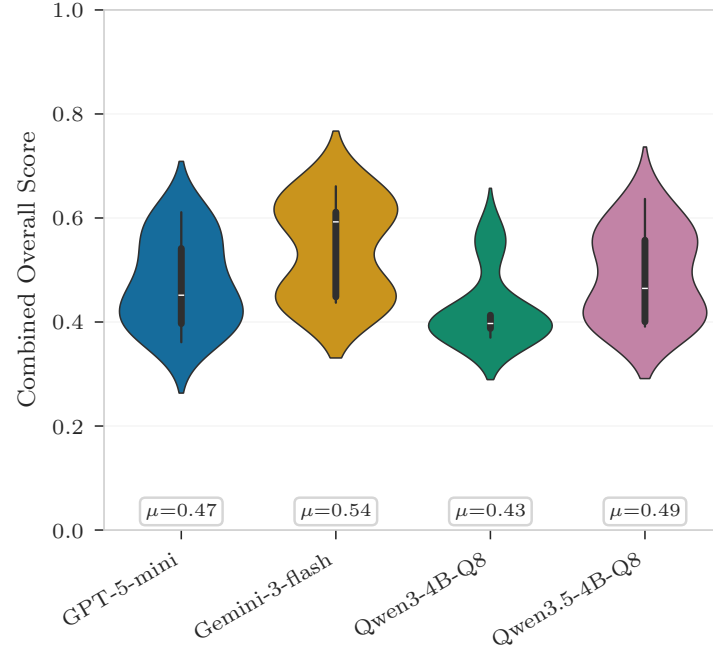


Figure 13: Combined overall score distributions for Photonics2D W-COND. The distributions have lower medians than the corresponding Beams2D results in Figure 4.

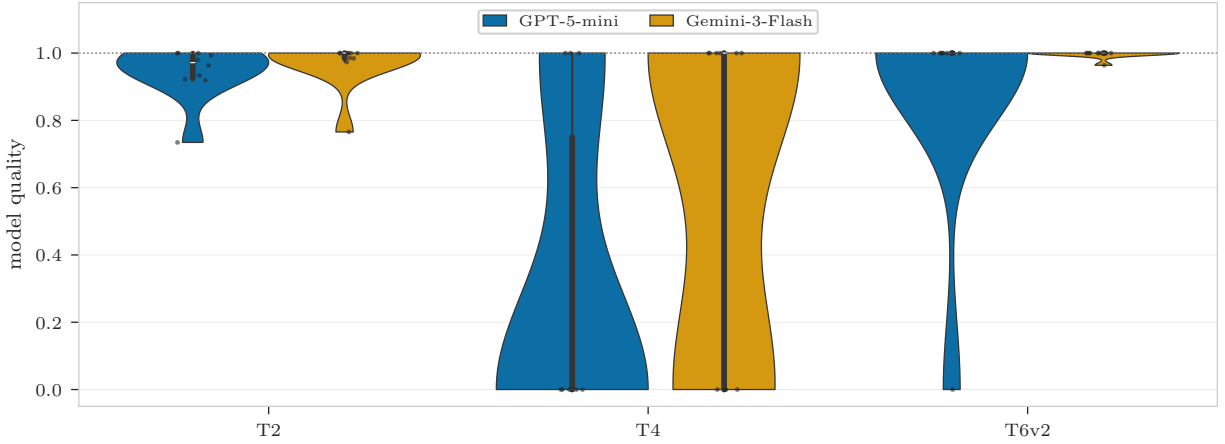


Figure 14: Per-seed model-quality scores by training-code tier and model backend on Photonics2D. Violin plots summarize ten seeds; boxes show the median and interquartile range, and points show individual seeds. Zero-valued T4 trials indicate checkpoints that the canonical evaluator could not load.

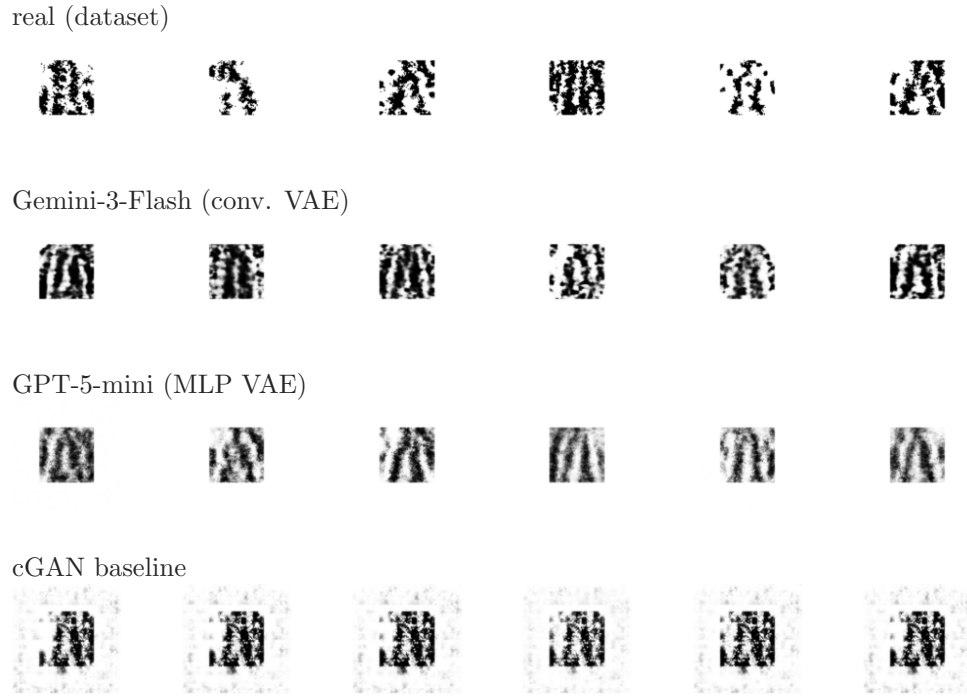


Figure 15: Photonics2D designs for seed 9. Columns correspond to fixed conditioning vectors containing the wavelengths and blur radius. Rows show the dataset design, the Gemini-3-Flash convolutional VAE, the GPT-5-mini fully connected VAE, and the reference cGAN. Designs are shown without thresholding.

Table 18: Raw per-metric scores (mean $\pm$ std) for all models and prompt styles on Beams2D. Most cells aggregate 15 runs (3 seeds $\times$ 5 samples); a few Qwen3.5-4B configurations have 13–14 runs due to JSON parsing errors. **Bold** = best model per metric within each prompt style. Metric abbreviations are defined in Appendix A of the main paper. The WT column is identically 0.00 across all models because our threshold-based STL extraction produces non-manifold meshes; this reflects a limitation of the post-processing pipeline rather than agent capability.

Style	Model	IoU $\uparrow$	PA $\uparrow$	Obj $\uparrow$	Constr $\uparrow$	Conn $\uparrow$	WT $\uparrow$	Tool Eff. $\uparrow$	TC $\uparrow$	DQ $\uparrow$	CO $\uparrow$
FULL	GPT-5-mini	0.40 $\pm$ 0.15	0.73 $\pm$ 0.08	0.52 $\pm$ 0.43	1.00 $\pm$ 0.00	0.67 $\pm$ 0.47	0.00 $\pm$ 0.00	0.72 $\pm$ 0.12	1.00 $\pm$ 0.00	0.54 $\pm$ 0.10	0.65 $\pm$ 0.07
	Gemini-3-Flash	0.39 $\pm$ 0.15	0.72 $\pm$ 0.08	0.49 $\pm$ 0.45	1.00 $\pm$ 0.00	0.73 $\pm$ 0.44	—	0.98 $\pm$ 0.08	0.93 $\pm$ 0.25	0.54 $\pm$ 0.09	0.69 $\pm$ 0.10
	Qwen3-4B	0.40 $\pm$ 0.15	0.73 $\pm$ 0.08	0.16 $\pm$ 0.33	1.00 $\pm$ 0.00	0.67 $\pm$ 0.47	—	0.67 $\pm$ 0.00	0.00 $\pm$ 0.00	0.49 $\pm$ 0.10	0.45 $\pm$ 0.06
	Qwen3.5-4B	0.40 $\pm$ 0.15	0.73 $\pm$ 0.08	0.37 $\pm$ 0.46	1.00 $\pm$ 0.00	0.67 $\pm$ 0.47	—	0.72 $\pm$ 0.05	0.73 $\pm$ 0.44	0.52 $\pm$ 0.09	0.59 $\pm$ 0.11
NATURAL	GPT-5-mini	0.39 $\pm$ 0.14	0.73 $\pm$ 0.05	0.04 $\pm$ 0.06	0.09 $\pm$ 0.12	0.75 $\pm$ 0.43	0.00 $\pm$ 0.00	0.75 $\pm$ 0.41	0.87 $\pm$ 0.34	0.37 $\pm$ 0.11	0.82 $\pm$ 0.30
	Gemini-3-Flash	0.34 $\pm$ 0.07	0.69 $\pm$ 0.05	0.10 $\pm$ 0.14	0.13 $\pm$ 0.11	0.67 $\pm$ 0.47	—	0.81 $\pm$ 0.33	1.00 $\pm$ 0.00	0.35 $\pm$ 0.07	0.88 $\pm$ 0.23
	Qwen3-4B	0.46 $\pm$ 0.18	0.72 $\pm$ 0.12	0.24 $\pm$ 0.32	0.05 $\pm$ 0.10	1.00 $\pm$ 0.00	—	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.44 $\pm$ 0.07	0.29 $\pm$ 0.05
	Qwen3.5-4B	0.47 $\pm$ 0.13	0.74 $\pm$ 0.09	0.13 $\pm$ 0.27	0.06 $\pm$ 0.11	0.91 $\pm$ 0.29	—	0.28 $\pm$ 0.44	0.33 $\pm$ 0.47	0.42 $\pm$ 0.07	0.48 $\pm$ 0.32
W-RAND	GPT-5-mini	0.40 $\pm$ 0.15	0.73 $\pm$ 0.08	0.52 $\pm$ 0.43	1.00 $\pm$ 0.00	0.67 $\pm$ 0.47	0.00 $\pm$ 0.00	0.77 $\pm$ 0.17	1.00 $\pm$ 0.00	0.54 $\pm$ 0.10	0.66 $\pm$ 0.07
	Gemini-3-Flash	0.40 $\pm$ 0.11	0.73 $\pm$ 0.06	0.35 $\pm$ 0.44	1.00 $\pm$ 0.00	0.73 $\pm$ 0.44	0.00 $\pm$ 0.00	0.80 $\pm$ 0.18	1.00 $\pm$ 0.00	0.52 $\pm$ 0.09	0.65 $\pm$ 0.09
	Qwen3-4B	0.39 $\pm$ 0.15	0.72 $\pm$ 0.08	0.48 $\pm$ 0.45	1.00 $\pm$ 0.00	0.87 $\pm$ 0.34	0.00 $\pm$ 0.00	0.63 $\pm$ 0.06	1.00 $\pm$ 0.00	0.55 $\pm$ 0.11	0.64 $\pm$ 0.07
	Qwen3.5-4B	0.40 $\pm$ 0.16	0.73 $\pm$ 0.08	0.38 $\pm$ 0.44	1.00 $\pm$ 0.00	0.71 $\pm$ 0.45	0.00 $\pm$ 0.00	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	0.53 $\pm$ 0.08	0.69 $\pm$ 0.06
W-DERIVED	GPT-5-mini	0.40 $\pm$ 0.15	0.73 $\pm$ 0.08	0.52 $\pm$ 0.43	1.00 $\pm$ 0.00	0.67 $\pm$ 0.47	0.00 $\pm$ 0.00	0.76 $\pm$ 0.14	1.00 $\pm$ 0.00	0.54 $\pm$ 0.10	0.65 $\pm$ 0.07
	Gemini-3-Flash	0.41 $\pm$ 0.13	0.74 $\pm$ 0.07	0.36 $\pm$ 0.43	0.93 $\pm$ 0.25	0.53 $\pm$ 0.50	0.00 $\pm$ 0.00	0.81 $\pm$ 0.14	1.00 $\pm$ 0.00	0.50 $\pm$ 0.11	0.64 $\pm$ 0.07
	Qwen3-4B	0.40 $\pm$ 0.15	0.73 $\pm$ 0.08	0.28 $\pm$ 0.40	1.00 $\pm$ 0.00	0.73 $\pm$ 0.44	0.00 $\pm$ 0.00	0.64 $\pm$ 0.07	0.47 $\pm$ 0.50	0.51 $\pm$ 0.08	0.53 $\pm$ 0.08
	Qwen3.5-4B	0.41 $\pm$ 0.16	0.73 $\pm$ 0.09	0.43 $\pm$ 0.46	1.00 $\pm$ 0.00	0.62 $\pm$ 0.49	0.00 $\pm$ 0.00	0.95 $\pm$ 0.18	0.85 $\pm$ 0.36	0.52 $\pm$ 0.09	0.66 $\pm$ 0.10
W-DISTRACT	GPT-5-mini	0.40 $\pm$ 0.15	0.73 $\pm$ 0.08	0.52 $\pm$ 0.43	1.00 $\pm$ 0.00	0.67 $\pm$ 0.47	0.00 $\pm$ 0.00	0.63 $\pm$ 0.08	1.00 $\pm$ 0.00	0.54 $\pm$ 0.10	0.63 $\pm$ 0.06
	Gemini-3-Flash	0.41 $\pm$ 0.14	0.74 $\pm$ 0.07	0.40 $\pm$ 0.43	1.00 $\pm$ 0.00	0.53 $\pm$ 0.50	0.00 $\pm$ 0.00	0.72 $\pm$ 0.06	1.00 $\pm$ 0.00	0.51 $\pm$ 0.11	0.63 $\pm$ 0.08
	Qwen3-4B	0.39 $\pm$ 0.15	0.73 $\pm$ 0.08	0.41 $\pm$ 0.44	1.00 $\pm$ 0.00	0.73 $\pm$ 0.44	0.00 $\pm$ 0.00	0.60 $\pm$ 0.00	1.00 $\pm$ 0.00	0.53 $\pm$ 0.10	0.61 $\pm$ 0.06
	Qwen3.5-4B	0.40 $\pm$ 0.15	0.73 $\pm$ 0.08	0.42 $\pm$ 0.45	1.00 $\pm$ 0.00	0.67 $\pm$ 0.47	0.00 $\pm$ 0.00	0.88 $\pm$ 0.12	0.93 $\pm$ 0.25	0.53 $\pm$ 0.10	0.66 $\pm$ 0.09
W-COND	GPT-5-mini	0.40 $\pm$ 0.15	0.73 $\pm$ 0.08	0.52 $\pm$ 0.43	1.00 $\pm$ 0.00	0.67 $\pm$ 0.47	0.00 $\pm$ 0.00	0.84 $\pm$ 0.16	0.93 $\pm$ 0.25	0.54 $\pm$ 0.10	0.66 $\pm$ 0.10
	Gemini-3-Flash	0.40 $\pm$ 0.16	0.73 $\pm$ 0.08	0.40 $\pm$ 0.43	0.93 $\pm$ 0.25	0.87 $\pm$ 0.34	0.00 $\pm$ 0.00	0.86 $\pm$ 0.17	0.87 $\pm$ 0.34	0.54 $\pm$ 0.10	0.65 $\pm$ 0.10
	Qwen3-4B	0.39 $\pm$ 0.15	0.72 $\pm$ 0.09	0.48 $\pm$ 0.44	1.00 $\pm$ 0.00	0.87 $\pm$ 0.34	0.00 $\pm$ 0.00	0.76 $\pm$ 0.15	0.40 $\pm$ 0.49	0.55 $\pm$ 0.10	0.57 $\pm$ 0.12
	Qwen3.5-4B	0.40 $\pm$ 0.15	0.73 $\pm$ 0.08	0.33 $\pm$ 0.41	1.00 $\pm$ 0.00	0.67 $\pm$ 0.47	0.00 $\pm$ 0.00	1.00 $\pm$ 0.00	0.60 $\pm$ 0.49	0.51 $\pm$ 0.10	0.62 $\pm$ 0.13
W-MULTI	GPT-5-mini	0.40 $\pm$ 0.15	0.73 $\pm$ 0.08	0.52 $\pm$ 0.43	1.00 $\pm$ 0.00	0.67 $\pm$ 0.47	0.00 $\pm$ 0.00	0.87 $\pm$ 0.13	0.93 $\pm$ 0.25	0.54 $\pm$ 0.10	0.66 $\pm$ 0.09
	Gemini-3-Flash	0.40 $\pm$ 0.15	0.73 $\pm$ 0.08	0.52 $\pm$ 0.43	1.00 $\pm$ 0.00	0.67 $\pm$ 0.47	0.00 $\pm$ 0.00	0.99 $\pm$ 0.05	1.00 $\pm$ 0.00	0.54 $\pm$ 0.10	0.70 $\pm$ 0.07
	Qwen3-4B	0.40 $\pm$ 0.15	0.73 $\pm$ 0.08	0.34 $\pm$ 0.41	1.00 $\pm$ 0.00	0.80 $\pm$ 0.40	0.00 $\pm$ 0.00	0.78 $\pm$ 0.05	1.00 $\pm$ 0.00	0.53 $\pm$ 0.11	0.65 $\pm$ 0.08
	Qwen3.5-4B	0.42 $\pm$ 0.14	0.74 $\pm$ 0.07	0.42 $\pm$ 0.42	1.00 $\pm$ 0.00	0.64 $\pm$ 0.48	0.00 $\pm$ 0.00	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	0.53 $\pm$ 0.10	0.69 $\pm$ 0.06

Table 19: Raw per-metric scores (mean $\pm$ std) for all models and prompt styles on Photonics2D. Most cells aggregate 15 runs (3 seeds $\times$ 5 samples); a few Qwen3.5-4B configurations have 13–14 runs due to JSON parsing errors. **Bold** = best model per metric within each prompt style. Metric abbreviations are defined in Appendix A of the main paper. The WT column is identically 0.00 across all models because our threshold-based STL extraction produces non-manifold meshes; this reflects a limitation of the post-processing pipeline rather than agent capability.

Style	Model	IoU $\uparrow$	PA $\uparrow$	Obj $\uparrow$	Constr $\uparrow$	Conn $\uparrow$	WT $\uparrow$	Tool Eff. $\uparrow$	TC $\uparrow$	DQ $\uparrow$	CO $\uparrow$
W-RAND	GPT-5-mini	0.32 $\pm$ 0.05	0.89 $\pm$ 0.01	0.00 $\pm$ 0.00	1.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.63 $\pm$ 0.06	1.00 $\pm$ 0.00	0.39 $\pm$ 0.02	0.53 $\pm$ 0.01
	Gemini-3-Flash	0.31 $\pm$ 0.05	0.89 $\pm$ 0.01	0.01 $\pm$ 0.03	1.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.97 $\pm$ 0.08	1.00 $\pm$ 0.00	0.39 $\pm$ 0.02	0.60 $\pm$ 0.02
	Qwen3-4B	0.31 $\pm$ 0.05	0.89 $\pm$ 0.01	0.04 $\pm$ 0.14	1.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.75 $\pm$ 0.00	1.00 $\pm$ 0.00	0.39 $\pm$ 0.03	0.56 $\pm$ 0.02
	Qwen3.5-4B	0.31 $\pm$ 0.06	0.89 $\pm$ 0.01	0.03 $\pm$ 0.10	1.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.73 $\pm$ 0.05	1.00 $\pm$ 0.00	0.39 $\pm$ 0.02	0.55 $\pm$ 0.01
W-DISTRACT	GPT-5-mini	0.31 $\pm$ 0.05	0.89 $\pm$ 0.01	0.02 $\pm$ 0.06	1.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.59 $\pm$ 0.07	1.00 $\pm$ 0.00	0.39 $\pm$ 0.02	0.52 $\pm$ 0.02
	Gemini-3-Flash	0.32 $\pm$ 0.05	0.89 $\pm$ 0.01	0.05 $\pm$ 0.15	1.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.72 $\pm$ 0.06	1.00 $\pm$ 0.00	0.40 $\pm$ 0.04	0.55 $\pm$ 0.03
	Qwen3-4B	0.31 $\pm$ 0.05	0.89 $\pm$ 0.01	0.04 $\pm$ 0.14	1.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.58 $\pm$ 0.04	1.00 $\pm$ 0.00	0.39 $\pm$ 0.03	0.52 $\pm$ 0.02
	Qwen3.5-4B	0.32 $\pm$ 0.06	0.89 $\pm$ 0.01	0.05 $\pm$ 0.19	1.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.82 $\pm$ 0.16	1.00 $\pm$ 0.00	0.40 $\pm$ 0.04	0.57 $\pm$ 0.05
W-COND	GPT-5-mini	0.32 $\pm$ 0.05	0.89 $\pm$ 0.01	0.06 $\pm$ 0.16	1.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.78 $\pm$ 0.16	0.40 $\pm$ 0.49	0.40 $\pm$ 0.03	0.47 $\pm$ 0.08
	Gemini-3-Flash	0.31 $\pm$ 0.05	0.89 $\pm$ 0.01	0.08 $\pm$ 0.17	1.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	1.00 $\pm$ 0.00	0.53 $\pm$ 0.50	0.40 $\pm$ 0.03	0.54 $\pm$ 0.09
	Qwen3-4B	0.31 $\pm$ 0.05	0.89 $\pm$ 0.01	0.04 $\pm$ 0.15	1.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.70 $\pm$ 0.05	0.20 $\pm$ 0.40	0.39 $\pm$ 0.03	0.43 $\pm$ 0.07
	Qwen3.5-4B	0.32 $\pm$ 0.05	0.89 $\pm$ 0.01	0.04 $\pm$ 0.15	1.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.82 $\pm$ 0.11	0.47 $\pm$ 0.50	0.39 $\pm$ 0.04	0.49 $\pm$ 0.08

Table 20: Unweighted components of the training-code score on Beams2D (mean $\pm$ std over ten seeds per cell). Column headings give each term’s weight in S_{tc} ; the final column is the composite under those weights. The components are reported separately so the composite can be recomputed under any alternative weighting. **Bold** = better model per component within each tier. Cells where model quality was unscorable are excluded from that column’s mean.

Tier	Model	MQ (.55) $\uparrow$	Code (.15) $\uparrow$	Job (.10) $\uparrow$	Dry (.10) $\uparrow$	Syn (.05) $\uparrow$	SLURM (.05) $\uparrow$	S_{tc} $\uparrow$
T2 (plumbing)	GPT-5-mini	0.74 $\pm$ 0.35	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	0.90 $\pm$ 0.30	0.85 $\pm$ 0.19
	Gemini-3-Flash	0.81 $\pm$ 0.31	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	0.90 $\pm$ 0.30	0.89 $\pm$ 0.18
T3 (training loop)	GPT-5-mini	0.68 $\pm$ 0.39	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	0.82 $\pm$ 0.22
	Gemini-3-Flash	0.74 $\pm$ 0.32	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	0.90 $\pm$ 0.30	0.85 $\pm$ 0.17
T4 (architecture)	GPT-5-mini	0.21 $\pm$ 0.40	1.00 $\pm$ 0.00	0.90 $\pm$ 0.30	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	0.55 $\pm$ 0.23
	Gemini-3-Flash	0.59 $\pm$ 0.36	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	0.90 $\pm$ 0.30	0.77 $\pm$ 0.19
T4-func	GPT-5-mini	0.74 $\pm$ 0.40	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	0.90 $\pm$ 0.30	1.00 $\pm$ 0.00	0.30 $\pm$ 0.46	0.81 $\pm$ 0.25
	Gemini-3-Flash	0.60 $\pm$ 0.36	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	0.70 $\pm$ 0.46	0.77 $\pm$ 0.21
T5 (whole module)	GPT-5-mini	0.40 $\pm$ 0.49	0.90 $\pm$ 0.30	1.00 $\pm$ 0.00	0.90 $\pm$ 0.30	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	0.65 $\pm$ 0.25
	Gemini-3-Flash	0.53 $\pm$ 0.40	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	0.90 $\pm$ 0.30	0.74 $\pm$ 0.22
T5-func	GPT-5-mini	0.00 $\pm$ 0.00	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	0.20 $\pm$ 0.40	0.41 $\pm$ 0.02
	Gemini-3-Flash	0.54 $\pm$ 0.41	0.90 $\pm$ 0.30	0.90 $\pm$ 0.30	0.90 $\pm$ 0.30	1.00 $\pm$ 0.00	0.70 $\pm$ 0.46	0.70 $\pm$ 0.30
T6v2 (open synth.)	GPT-5-mini	0.70 $\pm$ 0.35	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	0.20 $\pm$ 0.40	0.79 $\pm$ 0.20
	Gemini-3-Flash	0.88 $\pm$ 0.24	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	0.10 $\pm$ 0.30	0.89 $\pm$ 0.14

Table 21: Unweighted components of the training-code score on Photonics2D (mean $\pm$ std over ten seeds per cell). Column headings give each term’s weight in S_{tc} ; the final column is the composite under those weights. The components are reported separately so the composite can be recomputed under any alternative weighting. **Bold** = better model per component within each tier. Cells where model quality was unscorable are excluded from that column’s mean.

Tier	Model	MQ (.55) $\uparrow$	Code (.15) $\uparrow$	Job (.10) $\uparrow$	Dry (.10) $\uparrow$	Syn (.05) $\uparrow$	SLURM (.05) $\uparrow$	S_{tc} $\uparrow$
T2 (plumbing)	GPT-5-mini	0.94 $\pm$ 0.08	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	0.80 $\pm$ 0.40	0.96 $\pm$ 0.04
	Gemini-3-Flash	0.97 $\pm$ 0.07	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	0.80 $\pm$ 0.40	0.97 $\pm$ 0.04
T4 (architecture)	GPT-5-mini	0.30 $\pm$ 0.46	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	0.50 $\pm$ 0.50	0.59 $\pm$ 0.26
	Gemini-3-Flash	0.60 $\pm$ 0.49	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	0.90 $\pm$ 0.30	0.78 $\pm$ 0.28
T6v2 (open synth.)	GPT-5-mini	0.90 $\pm$ 0.30	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	0.00 $\pm$ 0.00	0.89 $\pm$ 0.17
	Gemini-3-Flash	1.00 $\pm$ 0.01	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	0.10 $\pm$ 0.30	0.95 $\pm$ 0.02